

GSPNet: Graph Spectral Projection Network Using Learnable Spectral Transformation

Yangli-ao Geng[✉], *Member, IEEE*, Yuxiao Dong[✉], *Senior Member, IEEE*, Wenzheng Feng[✉],
Qingyong Li[✉], *Member, IEEE*, and Jie Tang[✉], *Fellow, IEEE*

Abstract—Spectral graph convolutional networks (SGCNs) are one of the leading tools to handle learning tasks with graph structure. SGCNs leverage graph structure to define the graph spectral transformation (i.e., graph Fourier transformation) and pursue a good spectral filter in the spectral domain. For efficiency reasons, the pursued spectral filter is usually approximated by a polynomial of the normalized adjacency matrix in the spatial domain, referred to as the polynomial propagation of SGCNs. In this paper, we present a theoretical analysis on the propagation matrix defined by SGCNs and show that it actually almost always incurs a non-trivial propagation error. We also derive an explicit bound for this error. To mitigate this issue, we propose the Graph Spectral Projection Network (GSPNet) by learning a better spectral transformation, which endows GSPNet with potential to eliminate the error encountered by the polynomial propagation of SGCNs. Experimental results on commonly-used graph datasets suggest that GSPNet can learn a propagation matrix beyond the theoretically optimal polynomial propagation and generate promising results over existing SGCN models.

Index Terms—Graph neural networks, spectral graph convolutional networks, propagation matrix, matrix polynomial.

I. INTRODUCTION

RECENT years have witnessed a fast increase in data analytical tasks with graph structure, such as drug discovery [27], traffic forecasting [12], [32], and social analysis [39], [62]. Graph Neural Networks (GNNs) [56], [64] are recognized as the leading tool to handle these tasks. As a major branch of GNNs, Spectral Graph Convolutional networks (SGCNs) have received extensive attention owing to their solid theoretical foundation [8], [22], [34].

Typically, SGCNs consist of several SGCN layers, where each SGCN layer contains a feature transformation step followed by a feature propagation step [30], [56]. The former is usually implemented by a fully connected network layer aiming to

TABLE I
NOTATIONS USED IN THIS PAPER

Notation	Meaning
$\ \cdot\ _2$	ℓ_2 norm of a vector, or ℓ_2 operator norm (i.e., the largest singular value) of a matrix
$\ \cdot\ _F$	Frobenius norm of a matrix
$\mathcal{R}(\cdot), \text{tr}(\cdot)$	Range space, trace of a matrix
$\odot$	Hadamard (element-wise) product of homogeneous matrices
$\oplus$	Direct sum of matrices (i.e., binding them as diagonal blocks)
$\mathbb{S}_+^N$	Space of all $N \times N$ real non-negative symmetric matrices
$C, D, N \in \mathbb{R}$	Number of spectral bases, feature dimensions and samples
$\mathbf{A} = [a_{ij}] \in \mathbb{S}_+^N$	Adjacency matrix (either may or may not have self-loops)
$\mathbf{D}$	Diagonal degree matrix, i.e., $\mathbf{D} = \oplus_{i=1}^N d_i$, where $d_i = \sum_{j=1}^N a_{ij}$
$\hat{\mathbf{A}} \in \mathbb{S}_+^N$	Normalized adjacency matrix, i.e., $\hat{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$
$\mathbf{1}_N, \mathbf{I}_N$	all-one vector in $\mathbb{R}^N$, identity matrix in $\mathbb{R}^{N \times N}$

achieve informative feature representations for each node in the graph; the latter utilizes the graph structure to extract cross-node information for handling down-streaming tasks. Uniquely, the feature propagation step in the SGCN layer is performed by the spectral graph convolution operation [30], [66], which generalizes the spectral convolution from the Euclidean space to general spaces characterized by the graph [41]. Specifically, suppose that we are given an input graph with its normalized adjacency matrix $\hat{\mathbf{A}} \in \mathbb{R}^{N \times N}$, where N denotes the numbers of nodes in the graph. The spectral graph convolution first defines the graph spectral transformation (i.e., graph Fourier transformation) by leveraging the eigenvector matrix $\mathbf{P} \in \mathbb{R}^{N \times N}$ of $\hat{\mathbf{A}}$. Then a N -dimensional diagonal matrix Ψ is introduced as a filter in the graph spectral space. To be more specific, letting $\mathbf{X} \in \mathbb{R}^{N \times D}$ denote the corresponding node feature matrix, the spectral graph convolution can be formulated as $\mathbf{P} \Psi \mathbf{P}^\top \mathbf{X}$, where $\mathbf{P}^\top$ and $\mathbf{P}$ serve as the graph spectral transformation and its inverse, respectively.

However, since calculating the transformation matrix $\mathbf{P}$ relies on the computation-consuming eigen-decomposition of $\hat{\mathbf{A}}$, to breakthrough this computation bottleneck, many follow-up

Received 6 February 2023; revised 12 February 2026; accepted 6 April 2026. Date of publication 21 April 2026; date of current version 6 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grants 62276019, 62306028 and 62501043. Recommended for acceptance by P. Zhao. (*Corresponding author: Jie Tang.*)

Yangli-ao Geng and Qingyong Li are with the Key Laboratory of Big Data & Artificial Intelligence in Transportation (Ministry of Education), Beijing Jiaotong University, Beijing 100044, China (e-mail: gengyla@bjtu.edu.cn; liqy@bjtu.edu.cn).

Yuxiao Dong, Wenzheng Feng, and Jie Tang are with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: yuxiaod@tsinghua.edu.cn; fwz17@tsinghua.edu.cn; jietang@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TPAMI.2026.3685759

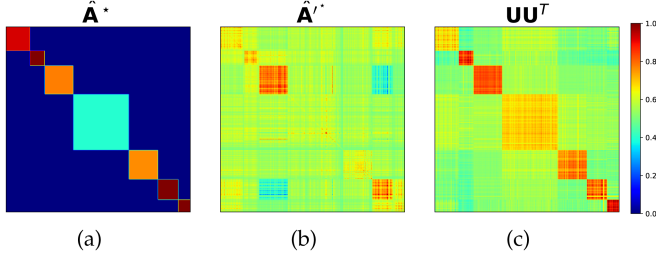


Fig. 1. Visualization of the propagation matrices on Cora. (a) the ideal ($\hat{\mathbf{A}}^*$) for the ground-truth class partitions, each of which is represented by one block; (b) the theoretically optimal (under the Frobenius norm metric) one that can be generated by existing SGCNs ($\hat{\mathbf{A}}'^*$); (c) the one learned by the proposed GSPNet method ($\mathbf{U}\mathbf{U}^\top$).

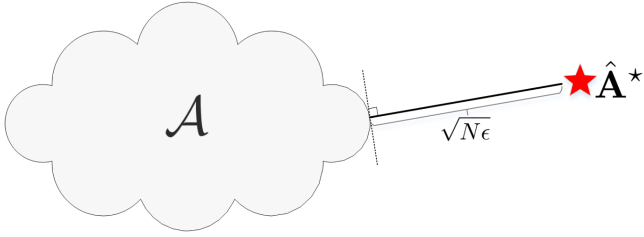


Fig. 2. Illustration for the gap between the SGCNs' propagation candidate set $\mathcal{A}$ and the ideal propagation matrix $\hat{\mathbf{A}}^*$. $\mathcal{A}$ consists of all the possible propagation matrices generated by existing SGCNs (see (4)). $\hat{\mathbf{A}}^*$ represents the theoretically optimal propagation matrix (see (8)). The gap between $\mathcal{A}$ and $\hat{\mathbf{A}}^*$ is at least $\sqrt{N}\epsilon$ under a mild condition (see Theorem 1).

works propose to directly approximate $\mathbf{P}\Psi\mathbf{P}^\top$ by leveraging various polynomial bases of $\hat{\mathbf{A}}$. These include classical approaches like Chebyshev polynomials [15] and Bernstein polynomials [22], as well as recent advancements utilizing wavelet bases [37], [61], universal polynomial bases for heterophily [25], and complex filters [14]. Notably, despite these sophisticated filter designs, the expressiveness of the propagation matrix generated by these approximation methods is still covered by that of the original SGCN one (i.e., $\mathbf{P}\Psi\mathbf{P}^\top$), considering that any polynomial of $\hat{\mathbf{A}}$ has an equivalent expression in the form of $\mathbf{P}\Psi\mathbf{P}^\top$. Furthermore, recent studies have pointed out that polynomial-based filters may face limitations regarding repeated eigenvalues [38], [43] and high-frequency signal capture [61].

In general, the SGCN methods fix the transformation $\mathbf{P}$ (determined by $\hat{\mathbf{A}}$), and aim to pursue a good spectral filter Ψ such that the propagated features $\mathbf{P}\Psi\mathbf{P}^\top\mathbf{X}$ have better representations, e.g., high intra-class similarity but low inter-class similarity. However, in practice, the input graph data and by extension $\hat{\mathbf{A}}$ may always contain extraneous information. Consequently, it is unknown whether the resultant propagation matrix $\mathbf{P}\Psi\mathbf{P}^\top$ can well characterize the ground-truth class partitions by adjusting a proper Ψ . In fact, we will show that in most practical cases the answer is negative. This limitation incurs an upper bound for the performance of the SGCN methods.

Formally, let us introduce an ideal normalized adjacency matrix $\hat{\mathbf{A}}^* \in \mathbb{R}^{N \times N}$ which can perfectly characterize the ground-truth class partitions (Cf. Fig. 1(a) for illustration and (8) for definition). We will later show that there almost always be a divergence between $\hat{\mathbf{A}}^*\mathbf{X}$ and $\mathbf{P}\Psi\mathbf{P}^\top\mathbf{X}$ —i.e., existing SGCNs

(Cf. Fig. 1(b))—no matter what values of Ψ are assigned. This divergence is referred to as the propagation error in this work. We will prove an explicit bound for the expected propagation error under a mild condition (see Theorem 1). Moreover, the propagation error can be gradually amplified as more SGCN layers are stacked, leading to the inaccurate aggregation of features between different classes [10], [65].

To overcome the above drawback, we propose the Graph Spectral Projection Network (GSPNet). Unlike recent works that focus on designing more complex filters (e.g., [58], [61]) within a fixed spectral domain, GSPNet aims to learn a better spectral transformation $\mathbf{P}$ itself. This endows GSPNet with the ability to eliminate the error encountered by the fixed propagation of standard SGCNs and approximate the ideal propagation $\hat{\mathbf{A}}^*$ (Cf. Fig. 1(c)). Experimental results on widely-used datasets suggest GSPNet outperforms existing SGCN baselines, including recent advanced models.

The contributions of this work are summarized as follows:

- 1) We present a theoretical analysis on the propagation step of SGCNs, formally characterizing the propagation error and showing that propagation matrices based on a fixed $\hat{\mathbf{A}}$ almost always incur non-trivial error.
- 2) We propose GSPNet to go beyond the propagation drawback of traditional SGCNs by learning the spectral transformation, proving its potential to learn an ideal propagation matrix.
- 3) Empirical studies demonstrate that GSPNet better approaches ideal propagation matrices and produces performance improvements over state-of-the-art spectral GNN baselines.

For ease of the ensuing presentation, some notations are defined in Table I. Unless stated otherwise, we use italic (e.g., a and A), boldface lowercase (e.g., $\mathbf{a}$) and boldface capital (e.g., $\mathbf{A}$) symbols to denote scalars, vectors and matrices, respectively.

II. RELATED WORKS

A. Spectral Graph Convolutional Networks

Graph Neural Networks (GNNs) [17], [44] generalize neural techniques into graph-structured data. As a major branch of GNNs, spectral graph convolution networks were first introduced in [8], the key design of which is to learn graph filters in the graph spectral domain. Following that, ChebNet [15] and GCN [30] extended this idea by approximating convolution filters with Chebyshev polynomials for acceleration. While early works focused on simplifying these operations [55] or addressing over-smoothing [10], [33], subsequent research has explored more complex filter approximations. For instance, GraphHeat [57], BernNet [22], and ARMA [4] utilize heat kernels, Bernstein polynomials, and rational filters, respectively, to enhance filtering capability.

In the last two years, the design of spectral filters has evolved significantly to address the limitations of standard polynomial bases. WaveNet [61] and WaveGC [37] introduce graph spectral wavelet convolutions via multi-resolution analysis and

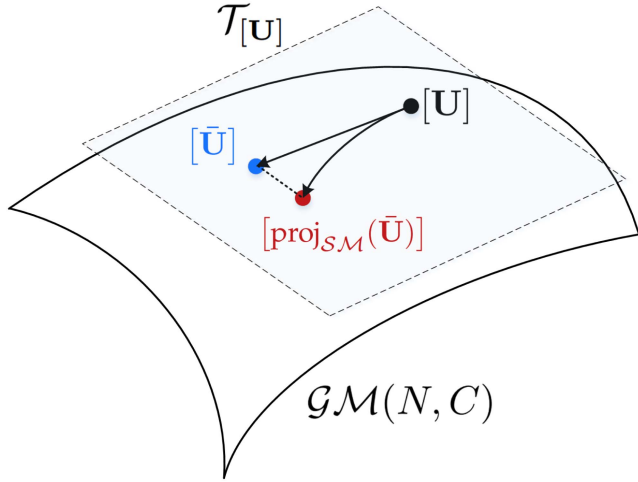


Fig. 3. Illustration for the gradient descent process on the Grassmann manifold $\mathcal{G}_M(N, C)$. It first moves from the current solution $[U]$ to $[U]$ along the negative Riemannian gradient direction $\text{grad}_{[U]} f$ (see (46) and (47)), where the path from $[U]$ to $[U]$ resides over the tangent space $\mathcal{T}_{[U]}$ at $[U]$. Then it projects $[U]$ from $\mathcal{T}_{[U]}$ back to $\mathcal{G}_M(N, C)$ and obtains a new solution $[\text{proj}_{S_M}(\bar{U})]$ (see (48)).

Chebyshev order decomposition to better capture high-frequency signals and non-stationary data. Addressing the challenge of heterophily, UniFilter [25] develops a universal polynomial basis that adapts to varying degrees of heterophily, while TFE-GNN [16] employs a triple filter ensemble to unify homophily and heterophily extraction. Furthermore, researchers have explored filters that transcend the polynomial constraint, such as SLOG [58] for inductive tasks, nonlinear spectral filters (NLSFs) [35] that are equivariant to graph functional shifts, and Arnoldi-based complex filters for graph contrastive learning [14]. Complementing these specific architectural innovations, Liao et al. [34] provide a unified spectral-oriented framework that benchmarks these diverse filter formulations, offering guidelines on balancing effectiveness with efficiency and memory consumption. Despite these advances, the propagation step in these methods largely remains a function of the fixed adjacency matrix, a limitation our work aims to address.

B. Theoretical Analysis of SGCNs

The expressive power of GNNs has been a central topic of theoretical investigation. Xu et al. [59] established a framework linking GNN expressiveness to the Weisfeiler-Lehman (WL) test. Specific to SGCNs, Balcilar et al. [3] and Wang et al. [53] analyzed spectral limits and connections to graph isomorphism testing. More recently, the impact of the graph spectrum on expressiveness has been rigorously examined. Lu et al. [38] and Qin et al. [43] theoretically establish that the number of distinguishable eigenvalues plays a pivotal role, showing that repeated eigenvalues in the Laplacian matrix significantly constrain the fitting capacity of polynomial filters. To address this, strategies like eigenvalue correction and perturbed sheaf Laplacians have been proposed. Zhang et al. [63] provide a unified framework for spectral invariant GNNs, proving they are bounded by the 3-WL test. Additionally, Jin et al. [26] extend the analysis of expressive

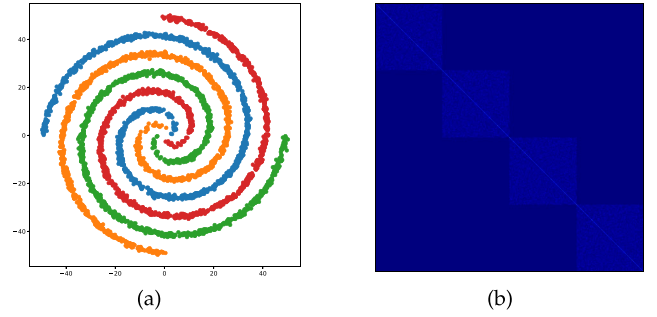


Fig. 4. Visualization for the synthetic Spiral dataset. (a) Feature distribution. (b) Pure adjacency matrix.

power to spectral-temporal GNNs. Beyond expressiveness, the optimization landscape has also been scrutinized; for example, Liu and Wang [36] reveal that the inherent differences between graph convolution parameters and feature transformation parameters lead to poorly conditioned optimization problems. Our work complements these theoretical studies by quantifying the propagation error incurred by the fixed spectral transformation.

C. Spectral Graph Methods

Spectral graph theory [13] underpins many applications in graph-based data mining. Spectral clustering [40], [45] partitions vertices by applying K-means to the eigenvectors of the Laplacian matrix and has close connections to graph-cut problems and random walks [48]. Variants have been proposed to improve robustness [11], [24] or preserve high-order proximity via truncated random walks [18], [42]. Recent developments continue to expand the scope of spectral methods. Wang and Wang [49] identify the information-theoretical threshold for exact recovery in semi-supervised learning using optimal spectral estimators. Kim and Oh [28] extend spectral analysis to bivariate graph signals, introducing tools for cross-spectral density and coherence. Unlike these methods which mainly focus on exploiting spectral techniques for mining existing graph structures, this paper investigates enhancing both the graph structure and the feature embedding by learning the spectral projection itself.

III. THE PROPAGATION ERROR IN SGCNS

We first introduce some basic concepts about the SGCN propagation matrix, and then present an analysis to show the limitations of the SGCN propagation matrix qualitatively. Finally, a provable propagation error bound of the SGCN propagation matrix is shown analytically.

A. The SGCN Propagation Matrix

Let $\mathbf{X} \in \mathbb{R}^{N \times D}$, $\hat{\mathbf{A}} \in \mathbb{S}_+^N$ and $\mathbf{W} \in \mathbb{R}^{D \times H}$ be the feature, normalized adjacency, and learnable weight matrices in SGCNs, respectively. Then one layer of SGCNs with a spectral filter $\Psi = \oplus_{i=1}^N \psi_i$ can be formulated as [8], [56]:

$$\mathbf{Y} = \sigma(\mathbf{P}\Psi\mathbf{P}^\top\mathbf{X}\mathbf{W}), \quad (1)$$

where $\sigma(\cdot)$ defines an element-wise non-linear function and $\mathbf{P}$ denotes the matrix of eigenvectors of $\hat{\mathbf{A}}$. However, due to the

$\mathcal{O}(N^3)$ computational complexity of eigenvalue decomposition (EVD), $\mathbf{P}\Psi\mathbf{P}^\top$ is usually approximated by a polynomial of $\hat{\mathbf{A}}$:

$$\mathbf{P}\Psi\mathbf{P}^\top \approx \sum_i \theta_i \hat{\mathbf{A}}^i = \mathbf{P} \left(\sum_i \theta_i \Lambda^i \right) \mathbf{P}^\top, \quad (2)$$

where $\Lambda = \oplus_{i=1}^N \lambda_i$ denotes the diagonal eigenvalue matrix of $\hat{\mathbf{A}}$ (i.e., $\mathbf{P}\Lambda\mathbf{P}^\top$ composes the EVD of $\hat{\mathbf{A}}$). The coefficients $\{\theta_i\}$ can be either predefined or (explicitly or implicitly) learnable. For example, GCN [30] sets θ_1 to 1 but the others to 0; the original SGCN [8] explicitly learns all $\{\theta_i\}$ in a fully data-driven manner; to eliminate the computational overhead and the risk of overfitting, many following methods [15], [22], [57] adopt truncated polynomials (e.g., Chebyshev polynomials and Bernstein polynomials) of $\hat{\mathbf{A}}$ as basis, with which a set of $\{\theta_i\}$ is implicitly learned.

If $\hat{\mathbf{A}}$ has N distinct non-zero eigenvalues $\lambda_1, \dots, \lambda_N$, there exists a unique solution to the following linear equations (with respect to $\theta_1, \theta_2, \dots, \theta_N$) [23]:

$$\begin{bmatrix} \lambda_1 & \lambda_1^2 & \dots & \lambda_1^N \\ \lambda_2 & \lambda_2^2 & \dots & \lambda_2^N \\ \vdots & \vdots & \ddots & \vdots \\ \lambda_N & \lambda_N^2 & \dots & \lambda_N^N \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_N \end{bmatrix} = \begin{bmatrix} \psi_1 \\ \psi_2 \\ \vdots \\ \psi_N \end{bmatrix}. \quad (3)$$

Under the above mild condition, (3) implies that $\mathbf{P}\Psi\mathbf{P}^\top$ can be equivalently expressed as a N -degree polynomial of $\hat{\mathbf{A}}$, i.e., for any real diagonal Ψ ,

$$\mathbf{P}\Psi\mathbf{P}^\top \in \mathcal{A} \triangleq \left\{ \sum_{i=1}^N \theta_i \hat{\mathbf{A}}^i \mid \theta_1, \dots, \theta_N \in \mathbb{R} \right\}. \quad (4)$$

$\mathbf{P}\Psi\mathbf{P}^\top$ can be viewed as the propagation matrix generated by SGCNs.

Equation (4) indicates that all SGCN propagation matrices can be covered by $\mathcal{A}$. Thus, discussions on the propagation of SGCNs boil down to the analysis of $\mathcal{A}$, which will be investigated from both the qualitative and quantitative aspects below.

B. Qualitative Analysis on $\mathcal{A}$

This subsection will present a qualitative analysis to show some limitations of the expressiveness of $\mathcal{A}$. Note that any $\hat{\mathbf{A}}' \in \mathcal{A}$ is a linear combination of the multi-hop neighbor adjacency matrices (e.g., $\hat{\mathbf{A}}^1, \hat{\mathbf{A}}^2, \dots$) derived by $\hat{\mathbf{A}}$. Therefore, $\hat{\mathbf{A}}'$ must be commutative with $\hat{\mathbf{A}}$ under the matrix multiplication, i.e.,

$$\hat{\mathbf{A}}' \in \mathcal{A} \implies \hat{\mathbf{A}}' \hat{\mathbf{A}} = \hat{\mathbf{A}} \hat{\mathbf{A}}'. \quad (5)$$

However, the matrices that are commutative with $\hat{\mathbf{A}}$ are only a small fraction of $\mathbb{S}_+^N$, where $\mathbb{S}_+^N$ is the space generated by all valid adjacency matrices. Moreover, the dimension of the matrix linear space generated by $\mathcal{A}$ is at most N , which is far less than that spanned by $\mathbb{S}_+^N$. Specifically, we have

$$\dim\{\mathcal{A}\} \leq N \ll N^2/2 + N/2 = \dim\{\mathbb{S}_+^N\}, \quad (6)$$

where $\dim\{\cdot\}$ denotes the dimension of the matrix linear space generated by $\cdot$.

Equation (6) indicates that the set $\mathcal{A}$ only covers a little part of the whole adjacency matrix space $\mathbb{S}_+^N$. Next, we will display the limitation of $\mathcal{A}$ by presenting an explicit expression bound.

C. A Provable Expressiveness Bound of $\mathcal{A}$

Since $\mathcal{A}$ covers all possible SGCN propagation matrices as analyzed in Section III-A, we first investigate the expression bound of $\mathcal{A}$, with which a provable error bound of the SGCN propagation matrix can be derived. To this end, we introduce the concept of an ideal adjacency matrix. Consider the ground-truth class partitions: $\mathcal{V}_1^*, \mathcal{V}_2^*, \dots, \mathcal{V}_C^*$, where $\mathcal{V}_i^*$ collects all the vertices of the class i and $\cup_{i=1}^C \mathcal{V}_i^* = \mathcal{V}$. Without loss of generality, suppose all the vertices have been sorted by their classes. Then the ideal adjacency matrix $\mathbf{A}^*$ for $\mathcal{V}_1^*, \mathcal{V}_2^*, \dots, \mathcal{V}_C^*$ is defined as:

$$\mathbf{A}^* = \bigoplus_{i=1}^C \mathbf{A}_i^*, \quad (7)$$

where $\mathbf{A}_i^* \triangleq \mathbf{1}_{|\mathcal{V}_i^*|} \mathbf{1}_{|\mathcal{V}_i^*|}^\top$ characterizes the complete sub-graph for $\mathcal{V}_i^*$. The normalized $\hat{\mathbf{A}}^*$ can be represented as

$$\hat{\mathbf{A}}^* = (\mathbf{D}^*)^{-1/2} \mathbf{A}^* (\mathbf{D}^*)^{-1/2}. \quad (8)$$

Property 1. Let $\hat{\mathbf{A}}^*$ be the ideal normalized adjacency matrix defined in (8). Then the EVD of $\hat{\mathbf{A}}^*$ has the form $\hat{\mathbf{A}}^* = \mathbf{U}^* \mathbf{U}^{*\top}$, where $\mathbf{U}^* = [u_{ij}^*] \in \mathbb{R}^{N \times C}$ is a semi unitary matrix defined by:

$$u_{ij}^* = \begin{cases} 1/\sqrt{|\mathcal{V}_i^*|} & \text{if } v_i \in \mathcal{V}_j^*, \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

Property 1 shows that the ideal adjacency matrix $\mathbf{A}^*$ are with a simple and low-rank spectral structure, based on which, we present the following theorem which gives an explicit expression bound of $\mathcal{A}$.

Theorem 1. Let $\hat{\mathbf{A}}^*, \hat{\mathbf{A}} \in \mathbb{S}_+^N$ be the ideal and the given normalized adjacency matrices, with their EVD $\hat{\mathbf{A}}^* = \mathbf{U}^* \mathbf{U}^{*\top}$ and $\hat{\mathbf{A}} = \mathbf{P}\Lambda\mathbf{P}^\top$, respectively. $\hat{\mathbf{A}}$ derives a propagation candidate set $\mathcal{A}$ which is defined in (4). Suppose that there exists a number $0 < \epsilon < C/N$ such that

$$\frac{1}{N} \sum_{i=1}^N \left\| \mathbf{U}^{*\top} \mathbf{p}_i \right\|_2^4 \leq \frac{C}{N} - \epsilon, \quad (10)$$

where $\mathbf{p}_i$ denotes the i -th column of $\mathbf{P}$. Then the Euclidean distance between $\hat{\mathbf{A}}^*$ and $\mathcal{A}$ is at least $\sqrt{N\epsilon}$, i.e.,

$$\inf_{\hat{\mathbf{A}}' \in \mathcal{A}} \|\hat{\mathbf{A}}^* - \hat{\mathbf{A}}'\|_F \geq \sqrt{N\epsilon}. \quad (11)$$

Proof. Note that any $\hat{\mathbf{A}}' \in \mathcal{A}$ has the same eigenvectors as $\hat{\mathbf{A}}$. Supposing it has EVD $\hat{\mathbf{A}}' = \mathbf{P}\Lambda'\mathbf{P}^\top$, then we have the following derivation:

$$\begin{aligned} & \|\hat{\mathbf{A}}^* - \hat{\mathbf{A}}'\|_F^2 \\ &= \|\hat{\mathbf{A}}^*\|_F^2 + \|\hat{\mathbf{A}}'\|_F^2 - 2\text{Tr}(\hat{\mathbf{A}}^{*\top} \hat{\mathbf{A}}') \\ &= C + \|\Lambda'\|_F^2 - 2\text{Tr}(\mathbf{U}^* \mathbf{U}^{*\top} \mathbf{P} \Lambda' \mathbf{P}^\top) \\ &= C + \|\Lambda'\|_F^2 - 2\text{Tr}(\mathbf{P}^\top \mathbf{U}^* \mathbf{U}^{*\top} \mathbf{P} \Lambda') \\ &= C + \|\lambda'\|_2^2 - 2\mathbf{1}_C^\top (\mathbf{R} \odot \mathbf{R}) \lambda', \end{aligned} \quad (12)$$

where $\lambda' \in \mathbb{R}^N$ collects all the diagonal elements of Λ' and $\mathbf{R} = \mathbf{U}^{\star\top} \mathbf{P} \in \mathbb{R}^{C \times N}$. (12) indicates that the infimum in the left side of (11) only depends on λ' , which is a convex optimization problem. Thus, letting $\nabla_{\lambda'} \|\hat{\mathbf{A}}^* - \hat{\mathbf{A}}'\|_F^2$ equal zero yields an optimal solution:

$$\lambda^* = (\mathbf{R} \odot \mathbf{R})^\top \mathbf{1}_C. \quad (13)$$

Substituting (13) back to (12) shows

$$\begin{aligned} & \inf_{\hat{\mathbf{A}}' \in \mathcal{A}} \|\hat{\mathbf{A}}^* - \hat{\mathbf{A}}'\|_F^2 \\ &= C + \|\lambda^*\|_2^2 - 2\mathbf{1}_C^\top (\mathbf{R} \odot \mathbf{R}) \lambda'^* \\ &= C - \|\lambda^*\|_2^2. \end{aligned} \quad (14)$$

Noting that (13) implies

$$\|\lambda^*\|_2^2 = \sum_{i=1}^N \lambda_i^{*2} = \sum_{i=1}^N \left\| \mathbf{U}^{\star\top} \mathbf{p}_i \right\|_2^4, \quad (15)$$

the condition (10) gives

$$\|\lambda^*\|_2^2 \leq C - N\epsilon. \quad (16)$$

Combing (14) and (16) leads to the desirable result (11). $\square$

Theorem 1 shows that even for the optimal propagation matrix chosen from $\mathcal{A}$, there may still be a gap $\sqrt{N}\epsilon$ between it and the underlying ideal adjacency matrix, as illustrated in Fig. 2.

Moreover, the condition (10) actually constraints the principal angle [1], [20] between the eigenvectors of $\hat{\mathbf{A}}$ and the C -dimensional subspace $\mathcal{R}(\mathbf{U}^*)$ to be large on average. It seems rather strong since the right side of (10) approaches zero (i.e., $\mathbf{p}_i$ is nearly orthogonal to $\mathcal{R}(\mathbf{U}^*)$) as N goes large. Nevertheless, it can be shown that the principal angle between $\mathbf{p}_i$ and $\mathcal{R}(\mathbf{U}^*)$ is highly concentrated around $\pi/2$ (i.e., they are very likely orthogonal indeed) when $C \ll N$ [1], [9], which is also validated in our experiments (see Section V-E). In other words, the condition (10) is fairly mild even for a very large N .

Theorem 1 enables us to analyze the divergence in propagation using the ideal adjacency matrix *versus* the propagation matrix chosen from $\mathcal{A}$. Specifically, for given $\hat{\mathbf{A}}^*$ and $\hat{\mathbf{A}}'$, we define the **expected propagation error** as

$$\mathbb{E}_{\mathbf{x} \sim \mathcal{US}(N)} \|(\hat{\mathbf{A}}^* - \hat{\mathbf{A}}')\mathbf{x}\|_2^2, \quad (17)$$

where $\mathcal{US}(N)$ denotes the uniform distribution over the surface of the unit sphere in $\mathbb{R}^N$.

Corollary 1. Suppose that the given adjacency matrix $\hat{\mathbf{A}}$ satisfies the condition presented in Theorem 1. Then the optimal propagation matrix $\hat{\mathbf{A}}^*$ generated from $\mathcal{A}$ incurs an **expected propagation error** of at least ϵ , where

$$\hat{\mathbf{A}}^* \in \arg \inf_{\hat{\mathbf{A}}' \in \mathcal{A}} \|\hat{\mathbf{A}}^* - \hat{\mathbf{A}}'\|_F. \quad (18)$$

Proof.

$$\mathbb{E}_{\mathbf{x} \sim \mathcal{US}(N)} \|(\hat{\mathbf{A}}^* - \hat{\mathbf{A}}^*)\mathbf{x}\|_2^2 = \mathbb{E}_{\mathbf{x} \sim \mathcal{US}(N)} \|\bar{\Lambda}\mathbf{x}\|_2^2$$

$$\begin{aligned} &= \mathbb{E}_{\mathbf{z} \sim \mathcal{UP}(N)} \sum_{i=1}^N z_i \bar{\lambda}_i^2 \\ &= \frac{1}{N} \|\hat{\mathbf{A}}^* - \hat{\mathbf{A}}^*\|_F^2 \\ &\geq \epsilon, \end{aligned} \quad (19)$$

where $\mathcal{US}(N)$ and $\mathcal{UP}(N)$ denote the uniform distribution over the unit sphere and the probability simplex in $\mathbb{R}^N$ [7], respectively; $\bar{\Lambda} = \oplus_{i=1}^N \bar{\lambda}_i$ is the diagonal eigenvalue matrix of $\hat{\mathbf{A}}^* - \hat{\mathbf{A}}^*$. $\square$

Loosely speaking, Corollary 1 states that even for the theoretically optimal propagation matrix chosen from $\mathcal{A}$, it almost always incurs a non-trivial propagation error. Recall that all the generated propagation matrices of SGCNs are covered by $\mathcal{A}$ (see (4)), this result further indicates the propagation error bound of SGCNs. Moreover, the propagation error can be gradually amplified as more SGCN layers are stacked. To eliminate this error in theory, one solution is to go beyond the polynomial propagation framework of SGCNs, which motivates GSPNet to be elaborated in the next section.

IV. THE GRAPH SPECTRAL PROJECTION NETWORK

A. The GSPNet Model

To go beyond the propagation error bound of the classic SGCN propagation matrix, we introduce the Graph Spectral Projection Network (GSPNet). Instead of learning the spectral filter (i.e., Ψ in (1)), GSPNet attempts to learn the spectral transformation (i.e., $\mathbf{P}$ in (1)) to eliminate the propagation error analyzed above.

However, directly learning $\mathbf{P}$ is computationally prohibitive due to its size of $\mathcal{O}(N^2)$. Nevertheless, Property 1 states that it is sufficient to approximate $\hat{\mathbf{A}}^*$ by using a tall-and-skinny orthonormal matrix $\mathbf{U} \in \mathbb{R}^{N \times C}$. Based on the above analysis, we update the SGCN layer in (1) with the following formulation:

$$\begin{aligned} & \min_{\substack{\mathbf{U} \in \mathbb{R}^{N \times C} \\ \mathbf{W} \in \mathbb{R}^{D \times C}}} L(\sigma(\mathbf{U}\mathbf{U}^\top \mathbf{X}\mathbf{W}), \mathbf{Y}) \\ & \text{s.t. } \mathbf{U}^\top \mathbf{U} = \mathbf{I}, \end{aligned} \quad (20)$$

where $\mathbf{Y}$ denotes the training label matrix. Note that $\mathbf{U}\mathbf{U}^\top \mathbf{X}$ can be viewed as projecting the columns of $\mathbf{X}$ onto the range subspace $\mathcal{R}(\mathbf{U})$, which explains the naming of GSPNet.

Theorem 2. The formulation (20) of GSPNet is with the theoretical potential to achieve the ideal propagation matrix $\hat{\mathbf{A}}^*$ defined in (8).

The proof would be clear in the light of Property 1. In addition, the given graph is expected to encompass some useful information. Therefore, GSPNet introduces a graph regularizer to mine knowledge from the given normalized adjacency matrix $\hat{\mathbf{A}}$:

$$\langle \mathbf{U}\mathbf{U}^\top, \hat{\mathbf{A}} \rangle = \text{tr}(\mathbf{U}^\top \hat{\mathbf{A}} \mathbf{U}), \quad (21)$$

which encourages the learned propagation matrix $\mathbf{U}\mathbf{U}^\top$ not to be far from $\hat{\mathbf{A}}$ under the metric of inner-product similarity. By taking (21) into the objective function, we reach the complete

formulation of GSPNet:

$$\begin{aligned} \min_{\substack{\mathbf{U} \in \mathbb{R}^{N \times C} \\ \mathbf{W} \in \mathbb{R}^{D \times C}}} & L(\sigma(\mathbf{U}\mathbf{U}^\top \mathbf{X}\mathbf{W}), \mathbf{Y}) - \alpha \text{tr}(\mathbf{U}^\top \hat{\mathbf{A}} \mathbf{U}) \\ \text{s.t. } & \mathbf{U}^\top \mathbf{U} = \mathbf{I}, \end{aligned} \quad (22)$$

where α is a hyper-parameter controlling the influence of $\hat{\mathbf{A}}$ on $\mathbf{U}$.

Although GSPNet can theoretically achieve $\mathbf{U}^*$ that characterizes the ideal adjacency matrix, it is unlikely to acquire the exact one in practice. Fortunately, we will show that GSPNet can achieve almost perfect classification for a good enough $\mathbf{U}$. To this end, we call an adjacency matrix as a *good enough* one $\mathbf{A}^\circ$ if the corresponding graph of $\mathbf{A}^\circ$ shares the same connected components (or partitions) with that of $\mathbf{A}^*$, but each connected component of $\mathbf{A}^\circ$ is not required to be completely connected.

Theorem 3. Suppose that $\mathbf{A}^\circ \in \mathbb{S}_+^N$ is a **good enough** adjacency matrix with $\hat{\mathbf{A}}^\circ \in \mathbb{S}_+^N$ being its normalized version, and $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the feature matrix. Let $\mathbf{U}^\circ = [u_{ij}^\circ] \in \mathbb{R}^{N \times C}$ be the matrix consisting of the top C eigenvectors of $\hat{\mathbf{A}}^\circ$. Then all the propagated features of the same class in $\mathbf{U}^\circ \mathbf{U}^{\circ\top} \mathbf{X}$ will distribute on a ray starting from $\mathbf{0}$. Furthermore, the direction vector of the j -th ray is the same as the weighted mean of the j -th class samples in $\mathbf{X}$, where the weight of the i -th sample is u_{ij}° .

Proof. The spectral graph theory claims that there exists an orthonormal matrix $\mathbf{Q} \in \mathbb{R}^{C \times C}$ such that the C columns of $\mathbf{U}^\circ \mathbf{Q}$ have exactly one-to-one correspondence with the C connected components $\mathbf{A}^\circ$: $\mathcal{V}_1^\circ, \dots, \mathcal{V}_C^\circ$ [48]. Without loss of generality, we suppose $\mathbf{Q} = \mathbf{I}_C$. Then there is

$$u_{ij}^\circ = \begin{cases} \sqrt{d_i^\circ / s_j^\circ} & \text{if } v_i \in \mathcal{V}_j^\circ, \\ 0 & \text{otherwise,} \end{cases} \quad (23)$$

where d_i° denotes the degree of v_i in $\mathbf{A}^\circ$ and $s_j^\circ = \sum_{v_i \in \mathcal{V}_j^\circ} d_i^\circ$. Note that

$$\mathbf{U}^\circ \mathbf{U}^{\circ\top} \mathbf{X} = \sum_{j=1}^C \mathbf{u}_j^\circ \mathbf{u}_j^{\circ\top} \mathbf{X}, \quad (24)$$

where $\mathbf{u}_j^\circ$ denotes the j -th column of $\mathbf{U}^\circ$. According to (23), for the j -th component $\mathcal{V}_j^\circ$, $\mathbf{u}_j^{\circ\top} \mathbf{X} \in \mathbb{R}^{1 \times D}$ can be viewed as the weighted mean of the features residing on $\mathcal{V}_j^\circ$. Furthermore, $\mathbf{u}_j^\circ (\mathbf{u}_j^{\circ\top} \mathbf{X}) \in \mathbb{R}^{N \times D}$ actually assigns the i -th row with the scaled weight mean $u_{ij}^\circ (\mathbf{u}_j^{\circ\top} \mathbf{X})$ for $v_i \in \mathcal{V}_j^\circ$, and masks the other rows with zeros. Therefore, $\{u_{ij}^\circ (\mathbf{u}_j^{\circ\top} \mathbf{X}) \mid v_i \in \mathcal{V}_j^\circ\}$ distribute on a ray with direction $\mathbf{u}_j^{\circ\top} \mathbf{X}$. Since $\mathbf{A}^\circ$ is **good enough**, all the vertices in $\mathcal{V}_j^\circ$ are exactly the ones that belong to the j -th class and the desired result follows. $\square$

According to Theorem 3, instead of the ideal $\mathbf{U}^*$, it is sufficient to learn a good enough $\mathbf{U}^\circ$ to endow GSPNet with strong classification capacity. Finding a $\mathbf{U}^\circ$ is much easier than demanding $\mathbf{U}^*$. Furthermore, in practice, the given normalized adjacency matrix $\hat{\mathbf{A}}$ can be usually viewed as a sum of $\hat{\mathbf{A}}^\circ$ and some noise matrix $\mathbf{N}$. Next, we will show that the graph regularizer in (22) is helpful to recover a pure $\mathbf{U}^\circ$ from a noisy case $\hat{\mathbf{A}}^\circ + \mathbf{N}$.

Theorem 4. Let $\hat{\mathbf{A}}^\circ$ and $\mathbf{U}^\circ$ be the matrices defined in Theorem 3. Let

$$\lambda_1^\circ \geq \lambda_2^\circ \geq \dots \geq \lambda_N^\circ$$

be the ordered eigenvalues of $\hat{\mathbf{A}}^\circ$ and $\mathbf{N} \in \mathbb{S}_+^N$ be a noise matrix. Suppose that

$$\|\mathbf{N}\|_2 < \min \left\{ \frac{\lambda_C^\circ - \lambda_{C+1}^\circ}{2}, \max \left\{ \lambda_{C+1}^\circ, \frac{\lambda_C^\circ}{3} \right\} \right\}. \quad (25)$$

Then the graph regularizer in (22) can perfectly recover the good enough $\mathbf{U}^\circ$, i.e.

$$\mathbf{U}^\circ = \underset{\substack{\mathbf{U} \in \mathbb{R}^{N \times C} \\ \mathbf{U}^\top \mathbf{U} = \mathbf{I}_C}}{\text{argmax}} \text{tr} \left(\mathbf{U}^\top (\hat{\mathbf{A}}^\circ + \mathbf{N}) \mathbf{U} \right). \quad (26)$$

Proof. The desired result is equivalent to proving that $\mathbf{U} = \mathbf{U}^\circ$ admits an optimal solution to the optimization problem shown in (26). We will prove it by studying the eigen-structure of $\hat{\mathbf{A}}^\circ + \mathbf{N}$. To begin with, suppose that the EVD of $\hat{\mathbf{A}}^\circ$ is given by

$$\hat{\mathbf{A}}^\circ = \mathbf{Q} \mathbf{\Lambda}^\circ \mathbf{Q}^\top = \sum_{i=1}^N \lambda_i^\circ \mathbf{q}_i \mathbf{q}_i^\top. \quad (27)$$

Then we have

$$\begin{aligned} \hat{\mathbf{A}}^\circ + \mathbf{N} &= \sum_{i=1}^N \lambda_i^\circ \mathbf{q}_i \mathbf{q}_i^\top + \mathbf{N} \\ &= \sum_{i=1}^N (\lambda_i^\circ + \mathbf{q}_i^\top \mathbf{N} \mathbf{q}_i) \mathbf{q}_i \mathbf{q}_i^\top + \tilde{\mathbf{N}} \\ &= \tilde{\mathbf{A}}_1^\circ + \tilde{\mathbf{A}}_2^\circ + \tilde{\mathbf{N}}, \end{aligned} \quad (28)$$

where

$$\tilde{\mathbf{A}}_1^\circ = \sum_{i=1}^C (\lambda_i^\circ + \mathbf{q}_i^\top \mathbf{N} \mathbf{q}_i) \mathbf{q}_i \mathbf{q}_i^\top = \sum_{i=1}^C \tilde{\lambda}_i^\circ \mathbf{q}_i \mathbf{q}_i^\top, \quad (29)$$

$$\tilde{\mathbf{A}}_2^\circ = \sum_{j=C+1}^N (\lambda_j^\circ + \mathbf{q}_j^\top \mathbf{N} \mathbf{q}_j) \mathbf{q}_j \mathbf{q}_j^\top = \sum_{j=C+1}^N \tilde{\lambda}_j^\circ \mathbf{q}_j \mathbf{q}_j^\top, \quad (30)$$

$$\begin{aligned} \tilde{\mathbf{N}} &= \mathbf{N} - \sum_{k=1}^N (\mathbf{q}_k^\top \mathbf{N} \mathbf{q}_k) \mathbf{q}_k \mathbf{q}_k^\top \\ &= \mathbf{N} - \mathbf{Q} (\mathbf{I}_N \odot (\mathbf{Q}^\top \mathbf{N} \mathbf{Q})) \mathbf{Q}^\top \\ &= \sum_{k=1}^N \tilde{\lambda}_k \mathbf{r}_k \mathbf{r}_k^\top. \end{aligned} \quad (31)$$

On the other hand, note that $\tilde{\mathbf{A}}_1^\circ$, $\tilde{\mathbf{A}}_2^\circ$ and $\tilde{\mathbf{N}}$ are orthogonal to each other, i.e.,

$$\langle \tilde{\mathbf{A}}_1^\circ, \tilde{\mathbf{A}}_2^\circ \rangle = \langle \tilde{\mathbf{A}}_1^\circ, \tilde{\mathbf{N}} \rangle = \langle \tilde{\mathbf{A}}_2^\circ, \tilde{\mathbf{N}} \rangle = 0. \quad (32)$$

Therefore, given any $\mathbf{U} \in \mathbb{R}^{N \times C}$ with $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_C$, we have

$$\text{tr} \left(\mathbf{U}^\top (\hat{\mathbf{A}}^\circ + \mathbf{N}) \mathbf{U} \right) = \text{tr} \left(\mathbf{U}^\top (\tilde{\mathbf{A}}_1^\circ + \tilde{\mathbf{A}}_2^\circ + \tilde{\mathbf{N}}) \mathbf{U} \right)$$

$$\begin{aligned}
&= \sum_{i=1}^C \tilde{\lambda}_i^\circ \|\mathbf{U}^\top \mathbf{q}_i\|_2^2 + \sum_{j=C+1}^N \tilde{\lambda}_j^\circ \|\mathbf{U}^\top \mathbf{q}_j\|_2^2 \\
&+ \sum_{k=1}^N \tilde{\lambda}_k \|\mathbf{U}^\top \mathbf{r}_k\|_2^2 \\
&= \sum_{i=1}^C \tilde{\lambda}_i^\circ \alpha_i + \sum_{j=C+1}^N \tilde{\lambda}_j^\circ \alpha_j \\
&+ \sum_{k=1}^N \tilde{\lambda}_k \beta_k,
\end{aligned} \tag{33}$$

where $\{\alpha_i\}_{i=1}^N$ and $\{\beta_i\}_{i=1}^N$ satisfy

$$0 \leq \alpha_i \leq 1, 0 \leq \beta_i \leq 1, (1 \leq i \leq N) \tag{34}$$

and

$$\sum_{i=1}^N \alpha_i + \sum_{i=1}^N \beta_i \leq C \tag{35}$$

due to the orthogonality (32). Thus solving (26) boils down to maximizing (33) under the constraints (34) and (35). Noting that $\mathbf{q}_i^\top \mathbf{N} \mathbf{q}_i \leq \|\mathbf{N}\|_2$ for any $1 \leq i \leq N$, the condition (25) thus implies that

$$\tilde{\lambda}_i^\circ \geq \lambda_C^\circ - \|\mathbf{N}\|_2 > \lambda_{C+1}^\circ + \|\mathbf{N}\|_2 \geq \tilde{\lambda}_j^\circ, \quad \forall 1 \leq i \leq C, C+1 \leq j \leq N, \tag{36}$$

and

$$\tilde{\lambda}_i^\circ \geq \lambda_C^\circ - \|\mathbf{N}\|_2 > \max\left\{\lambda_{C+1}^\circ, \frac{\lambda_C^\circ}{3}\right\} + \|\mathbf{N}\|_2 \geq 2\|\mathbf{N}\|_2 \geq \tilde{\lambda}_k, \quad \forall 1 \leq i \leq C, 1 \leq k \leq N, \tag{37}$$

where the last inequality in (37) is due to

$$\begin{aligned} \tilde{\lambda}_k &\leq \lambda_1(\mathbf{N}) + \lambda_1(\mathbf{Q}(\mathbf{I}_N \odot (\mathbf{Q}^\top \mathbf{N} \mathbf{Q})) \mathbf{Q}^\top) \\ &\leq 2\|\mathbf{N}\|_2. \end{aligned} \tag{38}$$

Combining (36) and (37) yields

$$\sum_{i=1}^C \tilde{\lambda}_i^\circ \geq \sum_{i=1}^C \tilde{\lambda}_i^\circ \alpha_i + \sum_{j=C+1}^N \tilde{\lambda}_j^\circ \alpha_j + \sum_{k=1}^N \tilde{\lambda}_k \beta_k, \tag{39}$$

implying that

$$\begin{aligned} \alpha_i &= 1, \alpha_j = 0, \beta_k = 0, \\ \forall 1 \leq i \leq C, C+1 \leq j \leq N, 1 \leq k \leq N, \end{aligned} \tag{40}$$

gives an optimal solution to maximizing (33) under the constraints (34) and (35). Recalling the definition of α_i in (33), (40) further implies

$$\|\mathbf{U}^\top \mathbf{q}_i\|_2 = 1, \quad \forall 1 \leq i \leq C. \tag{41}$$

$\mathbf{U} = \mathbf{U}^\circ$ satisfies (41) and thus admits an optimal solution to the optimization problem presented in (26). $\square$

Intuitively, Theorem 4 indicates that the GSPNet formulation (22) can yield a good enough spectral transformation when the noise “magnitude” $\|\mathbf{N}\|_2$ is not large compared with the eigen-gap $\lambda_C^\circ - \lambda_{C+1}^\circ$ of $\hat{\mathbf{A}}^\circ$. The eigen-gap further relies on the

connectivity strength within each connected component of $\hat{\mathbf{A}}^\circ$, i.e., stronger connectivity strength means a larger eigen-gap [48].

On the other hand, the final classification performance of GSPNet also depends on the directions of the C rays that $\mathbf{U}^\circ \mathbf{U}^{\circ\top} \mathbf{X}$ distributes on, i.e., the greater the divergence between these C directions, the better the classification performance. Furthermore, the divergence of the C directions relies on the feature distribution of $\mathbf{X}$, so it is necessary to introduce a feature transformation step. As a default choice, GSPNet uses a linear transformation $\mathbf{X} \mathbf{W}$ in (22). But we would like to emphasize that, depending on the raw data form, the feature transformation can be chosen for any proven network architectures such as ResNet [21].

B. Optimization for GSPNet

To optimize the objective function in (22), there are two variables $\mathbf{W}$ and $\mathbf{U}$ to be learned. $\mathbf{W}$ can be optimized with any gradient-based method such as Adam [29]. However, the optimization of $\mathbf{U}$ is non-trivial due to the orthonormal constraint in (22), as shown below.

First, note that (22) is invariant to orthonormal transformations with respect to $\mathbf{U}$. Specifically, denote $f(\mathbf{U}, \mathbf{W})$ as the objective function of (22). Then for any feasible solution $(\mathbf{U}, \mathbf{W})$ to (22) and orthonormal matrix $\mathbf{Q} \in \mathbb{R}^{C \times C}$, we have

$$f(\mathbf{U} \mathbf{Q}, \mathbf{W}) = f(\mathbf{U}, \mathbf{W}) \text{ and } (\mathbf{U} \mathbf{Q})^\top (\mathbf{U} \mathbf{Q}) = \mathbf{I}. \tag{42}$$

This redundancy can be eliminated by introducing the Grassmann manifold [2].

To begin with, the constraint set of (22) actually defines the so-called Stiefel manifold :

$$\mathcal{SM}(N, C) \triangleq \{\mathbf{U} \in \mathbb{R}^{N \times C} \mid \mathbf{U}^\top \mathbf{U} = \mathbf{I}_C\}. \tag{43}$$

On the other hand, consider the C -order orthogonal group $\mathcal{Q}(C) \triangleq \{\mathbf{Q} \in \mathbb{R}^{C \times C} \mid \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}\}$. With $\mathcal{Q}(C)$, we define an equivalent relation $\sim$ on $\mathcal{SM}(N, C)$ in the sense that $\mathbf{U}_1 \sim \mathbf{U}_2$ means that there exists a $\mathbf{Q} \in \mathcal{Q}(C)$ such that $\mathbf{U}_1 = \mathbf{U}_2 \mathbf{Q}$. The quotient space of $\mathcal{SM}(N, C)$ under this equivalent relation is called the Grassmann manifold [2]:

$$\mathcal{GM}(N, C) \triangleq \mathcal{SM}(N, C) / \mathcal{Q}(C). \tag{44}$$

$\mathcal{GM}(N, C)$ is a compact representation of the feasible solution set to the problem (22). Replacing the constraint set of (22) with $\mathcal{GM}(N, C)$ yields

$$\min_{\substack{[\mathbf{U}] \in \mathcal{GM}(N, C) \\ \mathbf{W} \in \mathbb{R}^{D \times C}}} f(\mathbf{U}, \mathbf{W}), \tag{45}$$

where $[\mathbf{U}] \triangleq \{\mathbf{U} \mathbf{Q} \mid \mathbf{Q} \in \mathcal{Q}(C)\}$ denotes the equivalent class derived by $\mathbf{U}$.

With the new formulation (45), we need to calculate the “gradient” with respect to $[\mathbf{U}]$. Given the Grassmann manifold $\mathcal{GM}(N, C)$, the Riemannian gradient [2], [50] on $\mathcal{GM}(N, C)$ with respect to $[\mathbf{U}]$ can be expressed as

$$\text{grad}_{[\mathbf{U}]} f = (\mathbf{I} - \mathbf{U} \mathbf{U}^\top) \nabla_{\mathbf{U}} f. \tag{46}$$

Algorithm 1: The Learning Algorithm for GSPNet.

Input: Feature matrix: $\mathbf{X} \in \mathbb{R}^{N \times D}$, Training labels: $\mathbf{Y}$,
 Normalized graph adjacency matrix: $\hat{\mathbf{A}} \in \mathbb{R}^{N \times N}$,
 Regularizer weight: α , Co-training loops: L

```

1 Initialize  $\mathbf{U} \in \mathbb{R}^{N \times C}$ ,  $\mathbf{W} \in \mathbb{R}^{D \times C}$ ,  $\hat{\mathbf{Y}} = \mathbf{Y}$ 
2 for  $l = 1, \dots, L$  do
    /* Stage 1: Feature view training via Adam */
    3
        
$$\min_{\mathbf{W} \in \mathbb{R}^{D \times C}} L(\sigma(\mathbf{XW}), \hat{\mathbf{Y}})$$

    /* Stage 2: Graph view training via (46)-(48) */
    4
        
$$\min_{[\mathbf{U}] \in \mathcal{GM}(N, C)} L(\sigma(\mathbf{UU}^\top \mathbf{XW}), \hat{\mathbf{Y}}) - \alpha \text{tr}(\mathbf{U}^\top \hat{\mathbf{A}} \mathbf{U})$$

    /* Stage 3: Pseudo-label assignment */
    5
        
$$\hat{\mathbf{Y}} \leftarrow \text{Pseudo-label Assignment}(\sigma(\mathbf{UU}^\top \mathbf{XW}))$$

6 end
    
```

where $\nabla_{\mathbf{U}} f$ denotes the normal gradient with respect to $\mathbf{U}$. Intuitively, (46) drops the part $\mathbf{UU}^\top \nabla_{\mathbf{U}} f$ from $\nabla_{\mathbf{U}} f$, since it does not affect the change of the value of f . Update $\mathbf{U}$ with the Riemannian gradient $\text{grad}_{[\mathbf{U}]} f$:

$$\bar{\mathbf{U}} \leftarrow \mathbf{U} - \beta \text{grad}_{[\mathbf{U}]} f, \quad (47)$$

where β denotes the learning rate. Then we need to project $\bar{\mathbf{U}}$ into $\mathcal{SM}(N, C)$:

$$\begin{aligned}
 \text{proj}_{\mathcal{SM}}(\bar{\mathbf{U}}) &\triangleq \underset{\mathbf{U} \in \mathcal{SM}(N, C)}{\text{argmin}} \|\bar{\mathbf{U}} - \mathbf{U}\|_F \\
 &= \mathbf{S}_L \mathbf{S}_R^\top,
 \end{aligned} \quad (48)$$

where $\mathbf{S}_L \in \mathcal{SM}(N, C)$ and $\mathbf{S}_R \in \mathcal{Q}(C)$ denote the left and the right orthonormal matrix of the singular value decomposition of $\bar{\mathbf{U}}$, respectively [24]. Fig. 3 illustrates the above gradient descent process on the Grassmann manifold $\mathcal{GM}(N, C)$.

Co-Training of GSPNet. In the above analyses, we illustrate the strong modeling potential of GSPNet in theory. In the extreme case, one can achieve any ground-truth partition by only choosing a proper $\mathbf{U}$, regardless of the learning of $\mathbf{W}$. However, the strong modeling potential also exacerbates the risk of over-fitting for GSPNet, incurring a gap between the practice performance and the potential theoretically promised by GSPNet. To bridge this gap, our idea is to introduce a divergence during the learning process between $\mathbf{U}$ and $\mathbf{W}$, which motivates a modified co-training strategy.

The original co-training strategy [5], [51] assumes that the description of a sample comprises two relatively independent views, either of which would be sufficient for learning given sufficient labeled data. The two learning algorithms are trained separately on each view. Then the prediction of each learning algorithm on the unlabeled data is viewed pseudo labels to enlarge the training set of the other.

For the graph node classification task, there are two naturally distinct views: the graph view characterized by $\hat{\mathbf{A}}$ and the

feature view $\mathbf{X}$. However, we empirically find that the simple pseudo-label transmission strategy between the two views fails to achieve the desired goal. A possible reason is that the pseudo labels yielded by the graph-view learning algorithm are with high confidence while that generated by the feature-view learning algorithm are with low confidence, as noted by Wang et al. [52]. To alleviate this issue, instead of exchanging information by pseudo labels between the two views, we update co-training by adopting an asymmetric information exchanging strategy. Specifically, from the feature view to the graph view, the learned feature transformation ($\mathbf{W}$) is passed; Reversely, the generated pseudo labels ($\hat{\mathbf{Y}}$) are transmitted, which has been validated to have high confidence [52]. The resultant training algorithm for GSPNet is listed in Algorithm 1.

V. EXPERIMENTS

Section V-A introduces the experimental setup. Section V-B evaluates GSPNet on semi-supervised node classification tasks and compares it with state-of-the-art GNN models. Section V-C studies the influence of the key hyper-parameters of GSPNet to its classification performance. Section V-D investigates the effect of the co-training strategy on GSPNet. Section V-E presents an empirical validation for Theorem 1. Section V-F visualizes the latent features GSPNet to better understand its learning behaviour. Section V-G investigates the training time and memory usage of GSPNet. Finally, we empirically evaluate the resilience of GSPNet against overfitting in Section V-H. We conduct experiments on a 64-bit Ubuntu 18.04 computer with 72 Intel 2.60 GHz CPUs, 256 GB memory, and 6 NVIDIA RTX 3090 GPUs. The code is available at <https://gitfront.io/t/gengyla/tEEB4jkZcqw3/GSPNet-TPAMI/>.

A. Experimental Setup

Baselines: We select a comprehensive range of GNN models as baselines, categorized into two types:

- General GNNs: GraphSage [19], GAT [47], SGC [55], APPNP [31] and DGC [54].
- Spectral GNNs: ChebNet [15], GCN [30], GraphHeat [57], FAGCN [6], S²GC [66], SLOG [58], UniFilter [25], WaveGC [37], WaveNet [61], and TFEgNN [16].

Datasets: We adopt eight commonly-used real datasets¹: Cora, Citeseer, Pubmed, Coauthor-Physics, Coauthor-CS, Amazon-Computers, Amazon-Photo, and Actor. In addition, we use a synthetic dataset Spiral to evaluate the classification performance of all the models under different noise levels. Fig. 4 visualizes its feature distribution and the corresponding pure graph adjacency matrix, where the pure graph adjacency matrix is generated by linking each sample with those that are in its 20 neighbors and belong to the same class. We inject $\tau\%$ noises² to the pure adjacency matrix to validate the robustness of all the methods. For Cora, Citeseer and Pubmed, we adopt the standard data split [60]. For the remaining datasets (Coauthor-Phy,

¹Available at https://pytorch-geometric.readthedocs.io/en/2.5.2/cheatsheet/data_cheatsheet.html.

²Randomly flip $\tau\%$ 1 to 0 and $\tau\%$ 0 to 1 in the pure adjacency matrix.

TABLE II
BENCHMARK DATASET STATISTICS

Dataset	Nodes	Edges	Train/Valid/Test	Classes	Features
Cora	2,708	5,429	140/500/1,000	7	1,433
Citeseer	3,327	4,732	120/500/1,000	6	3,703
Pubmed	19,717	44,338	60/500/1,000	3	500
Coauthor-Phy	34,493	247,962	100/150/34,243	5	8,415
Coauthor-CS	18,333	163,788	300/450/17,583	15	6,805
Amazon-Com	13,752	491,722	200/300/13,252	10	767
Amazon-Pho	7,650	238,162	160/240/7,250	8	745
Actor	7,600	30,019	100/150/7,350	5	932
Spiral	4,000	86,536	80/120/3,800	4	2

TABLE III

RESULTS OF NODE CLASSIFICATION (% TEST ACCURACY) ON EIGHT REAL DATASETS. THE TABLE GROUPS METHODS INTO GENERAL AND SPECTRAL CATEGORIES, FOLLOWED BY THE PROPOSED GSPNet. [†] INDICATES RESULTS ON CORA, CITESEER AND PUBMED ARE TAKEN FROM CORRESPONDING PAPERS. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD

Type	Methods	Cora	Citeseer	Pubmed	Coauthor-Phy	Coauthor-CS	Amazon-Com	Amazon-Pho	Actor
General	GraphSage	78.9±0.8	67.4±0.7	77.8±0.6	93.0±0.8	90.5±0.6	77.5±0.9	90.5±0.3	36.0±0.8
	GAT	83.0±0.7	72.5±0.7	79.0±0.3	92.5±0.9	87.2±0.4	41.0±10.6	61.7±3.0	26.1±0.2
	SGC	81.0±0.0	71.9±0.1	78.9±0.0	93.5±0.5	88.4±1.1	69.2±2.1	74.6±4.3	25.9±0.5
	APPNP	83.8±0.3	71.6±0.5	79.7±0.3	93.3±1.0	91.2±0.1	72.2±0.8	82.3±1.0	35.7±0.3
	DGC	83.3±0.0	73.3±0.1	80.3±0.1	92.9±0.8	89.8±0.2	66.0±2.3	81.4±0.6	25.5±0.1
Spectral	ChebNet [†]	81.2	69.8	74.4	66.0±19.9	90.6±0.4	77.2±0.5	89.3±0.4	35.3±0.5
	GCN [†]	81.5	70.3	79.0	92.8±1.0	91.5±0.8	80.6±0.8	89.3±1.0	31.1±1.1
	GraphHeat [†]	83.7	72.5	80.5	93.2±0.8	90.7±0.2	76.5±1.0	87.6±0.9	35.4±0.5
	FAGCN	84.1±0.5	72.7±0.8	79.4±0.3	92.3±1.4	91.1±0.2	80.2±0.4	89.9±0.3	36.3±0.5
	SSG	83.5±0.0	73.6±0.1	80.2±0.0	93.5±0.7	91.3±0.3	75.3±1.1	89.2±0.5	35.8±0.5
	SLOG	80.5±0.9	70.7±0.8	78.0±0.4	93.0±0.1	81.5±1.3	60.3±0.9	84.4±0.5	35.7±0.4
	UniFilter	81.1±0.7	69.9±0.3	79.0±0.3	92.2±1.8	92.4±0.9	80.8±0.6	89.4±0.5	35.5±0.4
	WaveGC	78.2±4.8	71.4±3.2	71.7±7.3	88.8±4.0	84.3±9.5	75.3±6.4	83.7±10.2	34.0±1.5
	WaveNet	79.4±0.4	69.3±0.9	77.3±0.5	91.7±0.2	86.5±0.1	75.7±0.5	88.7±0.3	35.4±0.2
	TFEGNN [†]	84.1±0.2	74.5±0.3	80.8±0.2	89.2±0.8	91.0±0.7	78.9±1.5	88.6±1.8	34.6±0.9
	GSPNet (Ours)	84.5±0.0	75.6±0.0	80.8±0.0	93.7±0.5	91.8±0.2	79.3±0.1	91.2±0.2	37.8±0.5

Coauthor-CS, Amazon-Com, Amazon-Pho, Actor) and Spiral, a random split (20/30 for training/validation, the remaining for test) is generated by using a given random seed. Table II shows the statistics for all the datasets under test. We conduct 50 trials with different random seeds and report mean and standard deviation of accuracy over the trials.

Parameter settings: GSPNet has two key parameters: the graph regularizer weight α and the number of co-training loops L . We tune α and L in the candidate sets of $\{0.1, 0.2, 0.3, \dots, 2.0\}$ and $\{1, 5, 15, 20, 25\}$ respectively. Note that we fix the number of the columns of $\mathbf{U}$ (see (22)) to the real class number C for each dataset, which endows GSPNet with a good balance between the performance and the memory usage as validated in Section V-C. For the baseline methods, the results on the datasets of Cora, Citeseer and Pubmed are taken from their official releases or respective papers. For the remaining datasets (Coauthor-Phy, Coauthor-CS, Amazon-Com, Amazon-Pho, Actor) and Spiral, we implement the methods ourselves. We follow the standard architecture in their original papers and tune their hyper-parameters in a rational range or use the recommendation if there any.

B. Semi-Supervised Node Classification

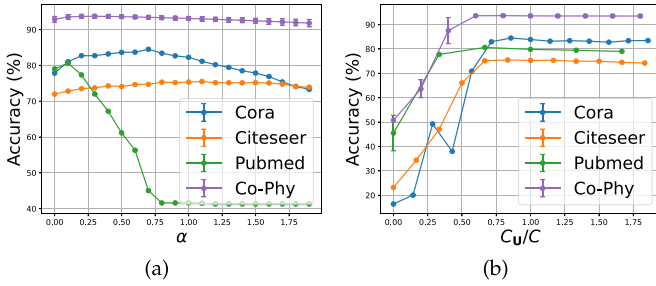
Table III demonstrates the classification accuracy of all the methods on eight real datasets. We observe that GSPNet achieves

superior results over both the general and the spectral GNN baselines on the majority of the datasets. Among the spectral baselines, UniFilter [25] and TFEGNN [16] exhibit strong competitiveness; UniFilter achieves the best performance on Coauthor-CS and Amazon-Com, while TFEGNN ties with GSPNet on Pubmed. However, GSPNet consistently outperforms these methods on the remaining five datasets (Cora, Citeseer, Coauthor-Phy, Amazon-Pho, and Actor). Notably, on the heterophilic Actor dataset, GSPNet surpasses the runner-up by a clear margin, validating its effectiveness in diverse structural settings. Moreover, GSPNet maintains a low standard deviation across most datasets, indicating stable performance compared to baselines like WaveGC [37], which shows higher variance on Coauthor-CS and Amazon-Pho.

Table IV displays the results of the spectral models on the Spiral dataset with the noise rate ($\tau\%$) changing from 0 to 70. $\tau = 0$ means using the pure adjacency matrix (see Fig. 4). While WaveNet [61] achieves perfect accuracy on the noise-free graph, its performance degrades significantly as noise is introduced. In contrast, GSPNet outperforms all baseline methods under noise rates from 10% to 70%. As the noise rate increases, the performance gap between GSPNet and the baseline methods becomes more pronounced. In particular, when $\tau = 70$, the given adjacency matrix is heavily polluted, causing a degeneration of the spectral transformation used by the spectral methods. Under this extreme condition, GSPNet achieves an around 9%

TABLE IV
 RESULTS OF NODE CLASSIFICATION (% TEST ACCURACY) ON SPIRAL WITH DIFFERENT NOISE RATES ($\tau\%$)

Method	Noise Rate ($\tau\%$)							
	0	10	20	30	40	50	60	70
GCN	78.5 $\pm$ 1.4	76.1 $\pm$ 1.7	71.4 $\pm$ 1.4	65.2 $\pm$ 8.7	61.1 $\pm$ 1.3	50.2 $\pm$ 8.9	36.5 $\pm$ 4.6	32.9 $\pm$ 2.3
GraphHeat	63.4 $\pm$ 10.7	60.9 $\pm$ 9.9	60.0 $\pm$ 8.9	58.9 $\pm$ 8.2	53.6 $\pm$ 9.8	52.7 $\pm$ 9.9	47.6 $\pm$ 9.3	34.4 $\pm$ 1.8
FAGCN	39.1 $\pm$ 4.6	37.2 $\pm$ 4.8	35.9 $\pm$ 3.4	35.3 $\pm$ 2.4	35.0 $\pm$ 2.5	34.9 $\pm$ 2.5	34.5 $\pm$ 2.5	34.3 $\pm$ 2.5
SSGC	61.9 $\pm$ 12.2	59.5 $\pm$ 11.5	59.1 $\pm$ 10.4	55.8 $\pm$ 11.9	52.0 $\pm$ 10.3	50.1 $\pm$ 9.5	49.4 $\pm$ 5.5	37.6 $\pm$ 8.1
SLOG	98.5 $\pm$ 0.2	50.4 $\pm$ 0.8	42.7 $\pm$ 1.3	32.7 $\pm$ 0.7	36.0 $\pm$ 0.7	35.0 $\pm$ 0.5	28.6 $\pm$ 0.5	26.2 $\pm$ 1.0
UniFilter	64.3 $\pm$ 2.2	53.1 $\pm$ 2.9	44.4 $\pm$ 1.9	41.7 $\pm$ 2.6	37.1 $\pm$ 5.3	35.7 $\pm$ 6.3	34.2 $\pm$ 2.8	31.4 $\pm$ 3.1
WaveGC	76.0 $\pm$ 3.2	48.3 $\pm$ 3.0	44.5 $\pm$ 2.3	37.0 $\pm$ 1.4	35.2 $\pm$ 1.7	34.2 $\pm$ 1.0	32.4 $\pm$ 1.7	33.0 $\pm$ 0.9
WaveNet	100.0$\pm$0.0	45.9 $\pm$ 3.2	49.3 $\pm$ 3.0	35.6 $\pm$ 1.8	29.4 $\pm$ 2.0	35.0 $\pm$ 2.0	28.3 $\pm$ 2.0	26.4 $\pm$ 0.8
TFEGNN	84.6 $\pm$ 2.2	57.0 $\pm$ 1.3	50.2 $\pm$ 1.5	44.4 $\pm$ 1.0	34.2 $\pm$ 1.0	34.0 $\pm$ 1.4	31.8 $\pm$ 1.0	30.3 $\pm$ 0.6
GSPNet	81.1 $\pm$ 14.7	79.6$\pm$11.9	74.6$\pm$12.0	70.7$\pm$9.7	65.8$\pm$8.6	59.4$\pm$5.7	54.4$\pm$6.2	46.4$\pm$3.0

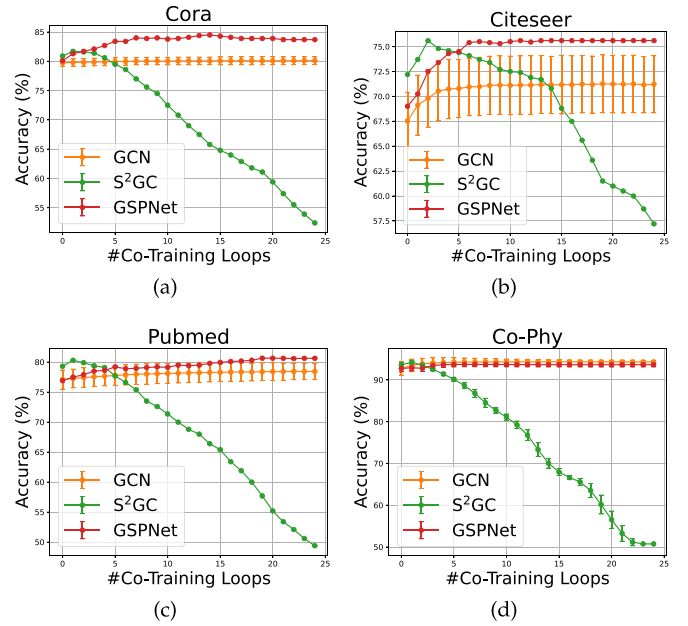

 Fig. 5. Classification accuracy (%) as functions of the key hyper-parameters (a) α (Cf. (22)) and (b) C_U/C (C_U and C denote the numbers of columns of $\mathbf{U}$ and true classes, respectively) for four real datasets.

lead over the second place SSGC. These results validate the superiority and the robustness of GSPNet's learnable spectral transformation compared to fixed or polynomial-based filters.

C. Study on Key Hyper-Parameters

This section investigates the influence of two key hyper-parameters on the performance of GSPNet. Fig. 5(a) visualizes the classification accuracy as a function of α (the weight of the graph regularizer, see (22)), while the other parameters are fixed. We observe that all curves exhibit an approximately uni-modal character, although their slopes vary. For Cora, Citeseer and Coauthor_Physics, their curves show a gentle trend in both ascent and descent. In contrast, the curve of Pubmed reaches its peak at 0.2 and then falls quickly. This may be because the graph of Pubmed fails to encompass sufficient category-relevant information (as illustrated in the third row in Fig. 7) such that large weights on the graph regularizer will incur a negative influence on the classification performance. The above results suggest that the hyper-parameter α should be tuned based on the quality of the given normalized adjacency matrix $\hat{\mathbf{A}}$. In general, better quality of $\hat{\mathbf{A}}$ encourages a larger α .

Fig. 5(b) plots the classification accuracy as a function of C_U/C , where C_U denoting the number of columns of $\mathbf{U}$ (see (22)) is the variable, and C representing the number of true classes is a constant for each dataset. We observe that all the curves share a very similar behavior: they increase monotonically as C_U increases and reach the plateau when $C_U/C = 1$ (i.e., $C_U = C$), and then they do not show evident changes


 Fig. 6. Effect of co-training on GCN, S^2GC and GSPNet on four real datasets. Each figure shows accuracy as a function of the number of co-training loops.

despite the increase in C_U . The above observation is consistent with our theoretical analysis presented in Section IV-A, that a C -rank $\mathbf{U}$ is sufficient to approximate the ideal propagation matrix $\hat{\mathbf{A}}^*$. In summary, it is recommended to set C_U equaling the number of true classes.

D. Effect of Co-Training

In this experiment, we investigate the behavior of GSPNet as the number of co-training loops changes. Moreover, we apply co-training to other two competitive SGCN models: GCN and S^2GC for comparison. Fig. 6 demonstrates the performance curve with the error bar of these models on four datasets. It can be observed that the accuracy of GCN (in orange) and GSPNet (in red) increases approximately monotonically as the number of co-training loops increases. In particular, they share a similar starting point, but GSPNet achieves a larger performance improvement than GCN with the rise in the number of co-training loops on all the datasets except Coauthor-Physics, on which

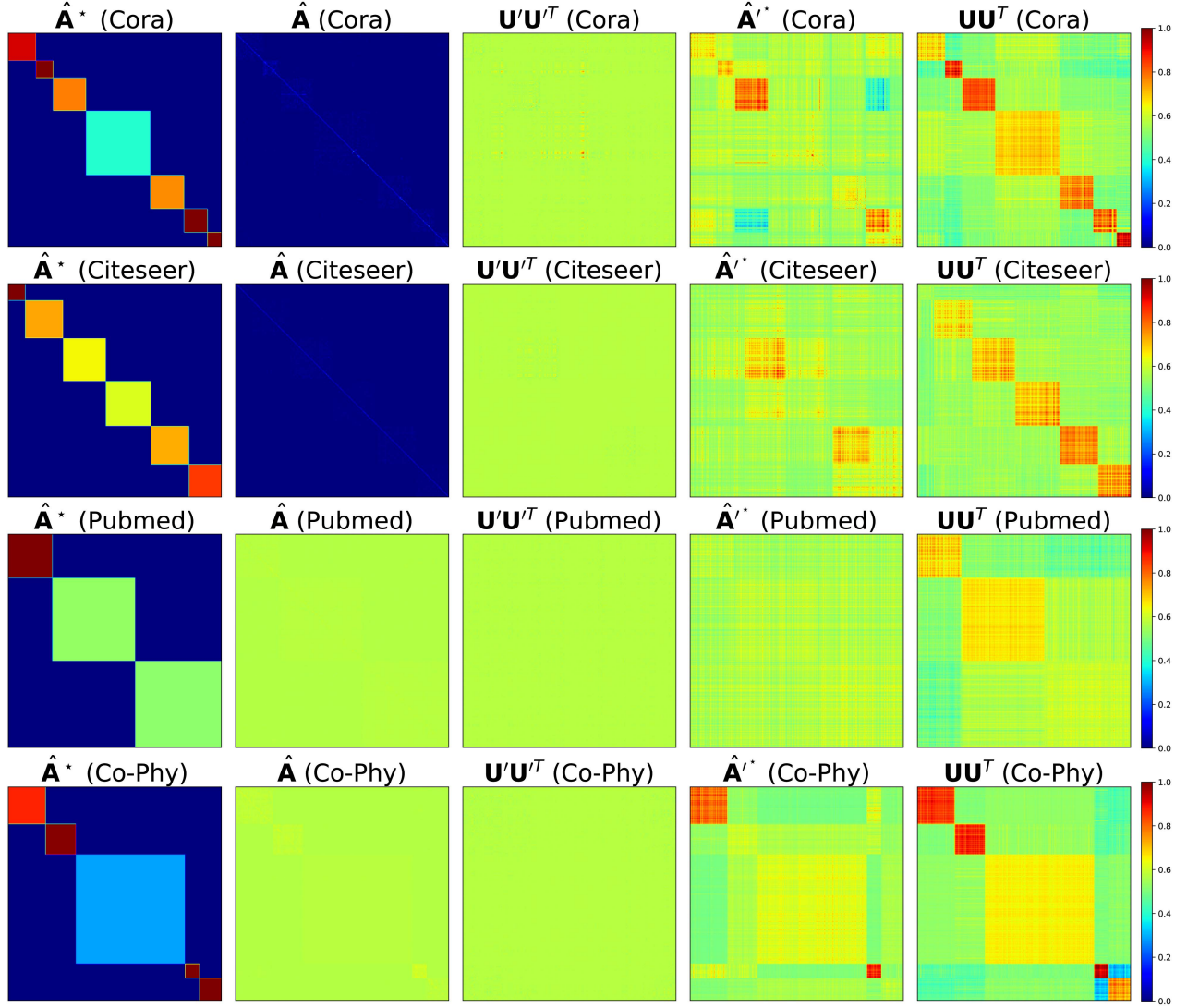


Fig. 7. Visualization of the key terms in Theorem 1 for four real datasets: Cora, Citeseer, Pubmed and Coauthor_Physics (from top to bottom). Columns from left to right are the heat maps of: $\hat{\mathbf{A}}^*$ (the ideal propagation matrix), $\hat{\mathbf{A}}$ (the given normalized adjacency matrix), $\mathbf{U}'\mathbf{U}'^T$ ($\mathbf{U}' \in \mathbb{R}^{N \times C}$ consists of the top C eigenvectors of $\hat{\mathbf{A}}$), $\hat{\mathbf{A}}'^*$ (the theoretical optimal $\hat{\mathbf{A}}'^*$ generated by SGCNs (see (18)) and $\mathbf{U}\mathbf{U}^T$ (learned by GSPNet).

GCN has a minor advantage over GSPNet. In addition, GSPNet has a very small standard deviation regardless of the number of co-training loops, which verifies the stability of GSPNet. In contrast, SGC achieves a similar improvement in the first few loops, but its accuracy peaked quickly and fell sharply after that. A possible reason is that under the co-training setting, $\mathcal{S}^2$ GC forces the same parameter $\mathbf{W}$ to fit different inputs (raw features and pre-propagated features [66]) in the two training views, which may cause an unstable learning process. In summary, the co-training strategy can generally improve the performance of GNN models, but the stopping loop may need to be carefully chosen depending on the specific model.

E. Empirical Validation for Theorem 1

In this experiment, we empirically validate the condition and the statement of Theorem 1. Table V reports specific values of the key terms in Theorem 1 on three real datasets.

TABLE V
SPECIFIC VALUES OF THE KEY TERMS IN THEOREM 1 ON FOUR REAL DATASETS

Terms in Theorem 1	Cora	Citeseer	Pubmed
C/N	0.002584	0.001803	0.000152
$\frac{1}{N} \sum_{i=1}^N \ \mathbf{U}^{\star\top} \mathbf{p}_i\ _2^4$	0.000513	0.000182	0.000034
$C/N - \frac{1}{N} \sum_{i=1}^N \ \mathbf{U}^{\star\top} \mathbf{p}_i\ _2^4$	0.002071	0.001621	0.000117
$\ \hat{\mathbf{A}}^* - \hat{\mathbf{A}}\ _F$	24.79795	32.81381	20.66164
$\ \hat{\mathbf{A}}^* - \mathbf{U}'\mathbf{U}'^T\ _F$	3.732699	3.459500	2.449210
$\ \hat{\mathbf{A}}^* - \hat{\mathbf{A}}'^*\ _F$	2.368697	2.322332	1.522010
$\ \hat{\mathbf{A}}^* - \mathbf{U}\mathbf{U}^T\ _F$	2.396875	2.586728	1.557847

Specifically, for the condition (10), we see that $\frac{1}{N} \sum_{i=1}^N \|\mathbf{U}^{\star\top} \mathbf{p}_i\|_2^4$ is far smaller than C/N , indicating that choosing any $\epsilon < C/N - \frac{1}{N} \sum_{i=1}^N \|\mathbf{U}^{\star\top} \mathbf{p}_i\|_2^4$ can satisfy this

condition. On the other hand, the lower part of Table V calculates the Euclidean distance between the potential ideal propagation matrix $\hat{\mathbf{A}}^*$ (see (8)) and several generated propagation matrices. They are the given normalized adjacency matrix $\hat{\mathbf{A}}$, the one generated by $\mathbf{U}' \in \mathbb{R}^{N \times C}$ which consists of the top C eigenvectors of $\hat{\mathbf{A}}$, the theoretical optimal $\hat{\mathbf{A}}^*$ generated by SGCNs (see (18)), and the $\mathbf{U}\mathbf{U}^\top$ learned by GSPNet. We observe that $\|\hat{\mathbf{A}}^* - \hat{\mathbf{A}}\|_F$ has large values on all four datasets, validating that the given $\hat{\mathbf{A}}$ is generally far from the ideal $\hat{\mathbf{A}}^*$. Furthermore, $\mathbf{U}'\mathbf{U}'^\top$ drastically reduces this distance from tens to single digits, since $\mathbf{U}'$ consists of low-frequency bases of $\hat{\mathbf{A}}$ which are usually believed to reduce noises. Furthermore, $\hat{\mathbf{A}}^*$ achieves the theoretical optimal approximation for $\hat{\mathbf{A}}^*$ under the SGCN framework. Nevertheless, there still be a non-trivial gap between $\hat{\mathbf{A}}^*$ and $\hat{\mathbf{A}}^*$, which validates the statements of Theorem 1. Finally, the bottom row of Table V displays that the $\mathbf{U}\mathbf{U}^\top$ learned by GSPNet is very close to $\hat{\mathbf{A}}^*$.

In addition, Fig. 7 visualizes the propagation matrices of $\hat{\mathbf{A}}^*$ (the ideal propagation matrix), $\hat{\mathbf{A}}$ (the given normalized adjacency matrix), $\mathbf{U}'\mathbf{U}'^\top$ ($\mathbf{U}' \in \mathbb{R}^{N \times C}$ consists of the top C eigenvectors of $\hat{\mathbf{A}}$), $\hat{\mathbf{A}}^*$ (the theoretical optimal $\hat{\mathbf{A}}^*$ generated by SGCNs (see (18)) and $\mathbf{U}\mathbf{U}^\top$ (learned by GSPNet). We observe that $\hat{\mathbf{A}}^*$ perfectly characterizes the ground-truth class partitions. The second and third columns demonstrate that the given $\hat{\mathbf{A}}$ and its pure low-rank approximation $\mathbf{U}'\mathbf{U}'^\top$ provide little evidence for the node classification task. Moreover, the fourth column displays that although $\hat{\mathbf{A}}^*$ achieves the theoretical optimal approximation under the SGCN framework, it fails to fully capture the underlying category information. In contrast, the last column shows that the $\mathbf{U}\mathbf{U}^\top$ learned by GSPNet seizes the main category structure, which supports its superior performance shown in Section V-B.

F. Feature Embedding Visualization

To better understand the learning behavior of the GSPNet through the co-training process, we visualize the latent features (before the softmax operation) at different co-training loops in Fig. 8 using TSNE [46]. From an overall perspective, the latent features of different categories become stronger discriminative as the number of co-training loops increases, indicating the effectiveness of the co-training strategy for GSPNet. Furthermore, we observe the feature distribution after the graph view training (GVL) demonstrates a larger classification margin than that after the feature view training (FVL). The underlying reason for this phenomenon is that the learnable spectral transformation design provides strong modeling potential, which enables GSPNet to learn more non-linear mapping to enlarge the margin of the feature distribution between different classes. This priority also benefits the FVL of the next co-training loop via the pseudo-label transferring mechanism. In summary, the FVL and GVL allow GSPNet to alternatively enhance the discriminativeness of the feature representation from the two complementary views, which eliminates the over-fitting risk for GSPNet and ensures its superior performance.

G. Study on Training Time and Memory Usage

This part is dedicated to an empirical investigation of GSPNet's actual running time and storage overhead. Our study involves two distinct experiments: the first is a comparative study between GSPNet and various state-of-the-art methods using real-world datasets, while the second experiment examines GSPNet's scalability with simulated datasets.

In the first experiment, we compare GSPNet against the five classical SGCN baselines. The comparison is performed on four real-world datasets. Each model is configured with a two-layer architecture and a hidden layer size of 64. We standardize the training process with 500 epochs and a learning rate of 0.2.

As illustrated in Fig. 9, all methods exhibit training times and memory usage within the same order of magnitude. Among all the evaluated methods, FAGCN shows the longest training time across all datasets. This extended duration is primarily attributed to its intricate operations: during execution, FAGCN continuously decomposes data into high-frequency and low-frequency components relative to the graph and integrates them using a gating mechanism. In contrast, GraphHeat and S²GC recorded the shortest training times but incurred greater memory overhead. This is mainly due to the pre-propagation technique they utilize, where node features are propagated across the graph using a convolutional kernel, followed by a multi-layer perceptron for processing these propagated features. This technique decouples graph propagation from feature transformation, thereby simplifying the gradient backpropagation process and reducing training duration. However, the pre-propagation of features necessitates the storage of multiple feature copies, escalating memory requirements. ChebNet and GCN adopt an alternating design between graph propagation and feature transformation, achieving a balance between training time and memory consumption. Regarding GSPNet, it does not exhibit a significant increase in storage overhead compared to other methods. This efficiency is attributed to GSPNet's "low-rank" representation of the spectral transformation $\mathbf{U} \in \mathbb{R}^{N \times C}$, which is typically much smaller than the original node feature matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ (i.e., $C \ll D$), thereby mitigating substantial memory costs. Concerning training time, GSPNet shows competitive performance on smaller datasets such as Cora and Citeseer, but it is slightly slower on larger datasets like Pubmed and Co-Phy. Note that the larger gap on Pubmed may be caused by hardware behavior, which will be detailed in the second experiment. Nonetheless, GSPNet's training duration remains comparable to the other five models. Next, we will provide a more exhaustive analysis of the variations in GSPNet's training time and memory overhead in relation to the dataset scale.

In the second experiment, we focus on evaluating the scalability of GSPNet, both in terms of computational and memory requirements, on a series of simulated Gaussian datasets. We examine the impact of three principal factors: the number of nodes (N), the dimension of features (D), and the count of spectral bases (C). Our study involves tracking the training duration and GPU memory usage of GSPNet under three settings:

Setting 1: $N \in \{2^{10}, 2^{11}, \dots, 2^{20}\}$, $D = 2^7$, $C = 2^3$.



Fig. 8. TSNE visualization of latent features (before the softmax operation) for four real datasets: Cora, Citeseer, Pubmed and Coauthor_Physics (Co-Phy) (corresponding four parts from top to bottom). For the two rows of each part, the first row and the second row plot the latent features after the feature view learning (FVL) and the graph view learning (GVL), respectively. The six columns from left to right correspond to the cases of the co-training loop equaling 1, 5, 10, . . . , 25.

Setting 2: $N = 2^{15}, D \in \{2^6, 2^7, \dots, 2^{16}\}, C = 2^3$.

Setting 3: $N = 2^{15}, D = 2^7, C \in \{2^1, 2^2, \dots, 2^{10}\}$.

(49)

We assess the GSPNet model, as defined in (53), varying the layer number L from 1 to 4. For configurations where $L \geq 2$, we set the hidden layer size to 128. The total number of training epochs was fixed at 500. In these experiments, we repeat each

scenario 10 times with different random seeds and report the mean and the standard deviation in Fig. 10.

Our observations indicated that both computational and memory requirements of all networks escalate with the increase in the size of the independent variables. Regarding training time, we observed an initial slow and steady increase, followed by a more pronounced surge at a certain point. This trend can be attributed to the GPU's surplus parallel computing capabilities, which effectively process additional data without significant

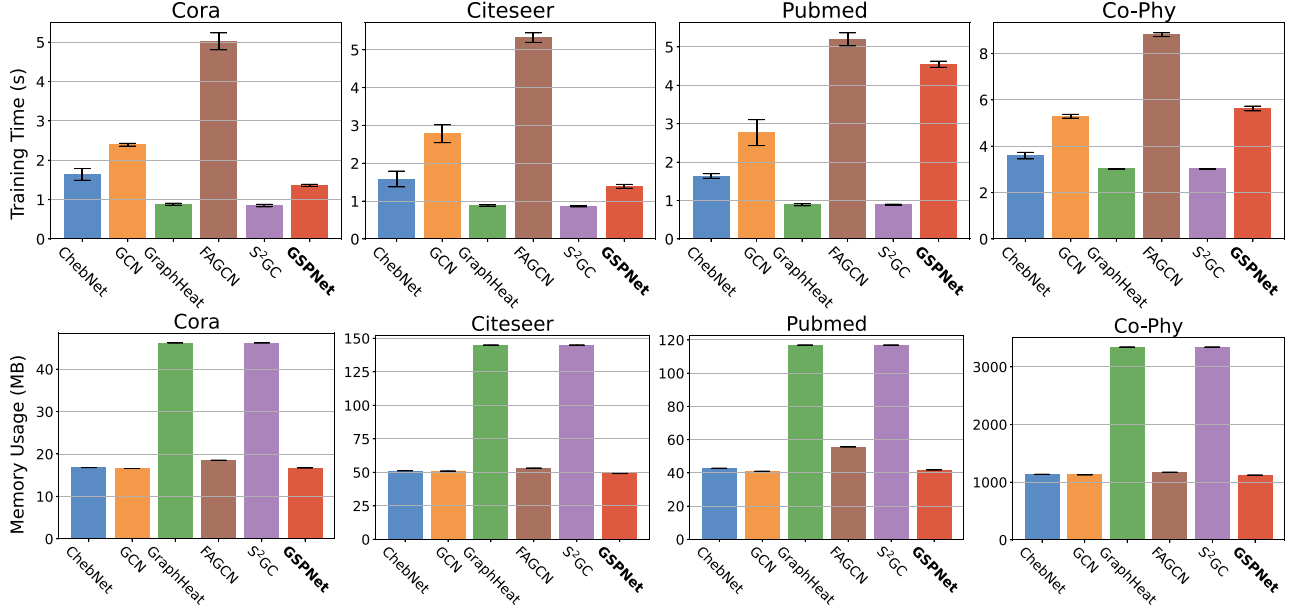


Fig. 9. The training time (in seconds) and GPU memory usage (in megabytes) of the proposed GSPNet across four real-world datasets, compared with five SGCN baselines.

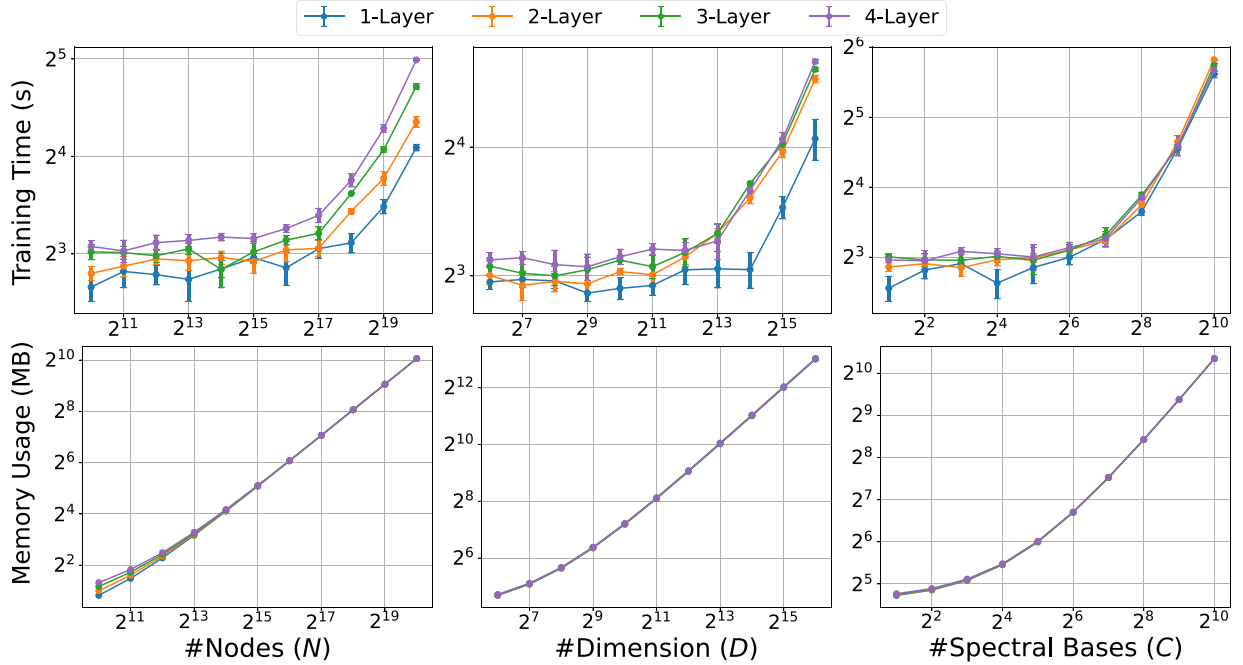


Fig. 10. Training time (in seconds) and GPU memory usage (in megabytes) of GSPNet (with layers 1 to 4) as functions of the number of nodes N , data dimension (D), and the count of spectral bases (C). The detailed experimental settings are provided in (49). We repeat each experiment 10 times with different random seeds and report the mean and standard deviation.

increases in computational time at lower data scales. Beyond a certain data volume, the GPU's parallel processing capacity reaches saturation, leading to a linear escalation in computation time relative to the data size. The detailed findings for the three variables under study are as follows:

- **Training Time versus Number of Nodes (N):** Illustrated in the top-left plot of Fig. 10, the training time for all four network configurations exhibits a similar pattern. They

started with a mild increase, but upon reaching a critical point at 2^{17} nodes, they displayed a linear growth trend with an approximate slope of 1. In particular, training a 4-layer GSPNet on a graph with one million nodes took only about 32 seconds on our test device.

- **Training Time versus Feature Dimension (D):** A noticeable disparity in training time, especially in high-dimensional settings (i.e., $D \geq 2^{13}$), was observed between the 1-layer

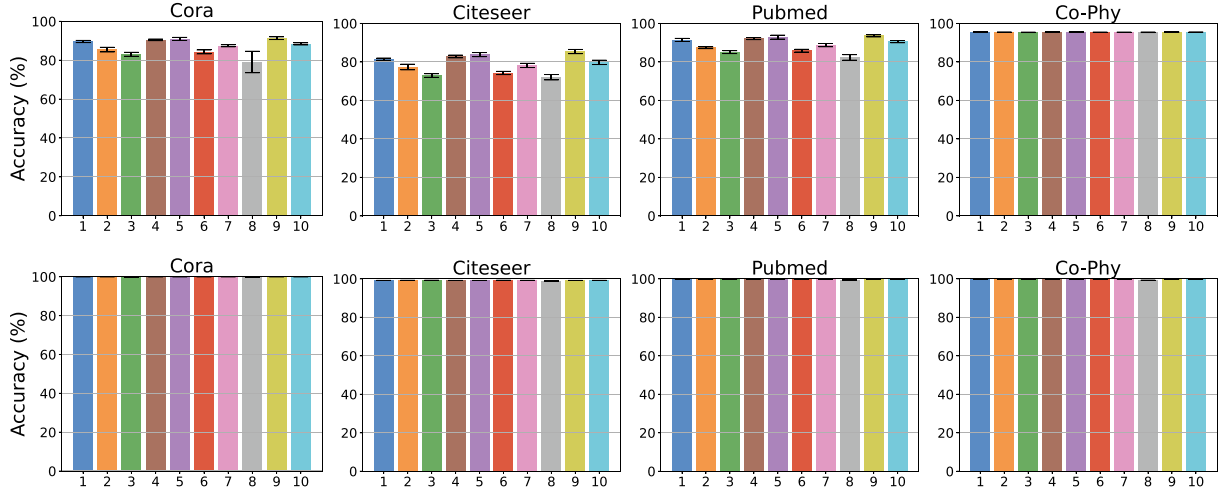


Fig. 11. **Top Row:** 5-fold label-scarce cross-validation (training on 1 fold, testing on 4), designed for semi-supervised settings. **Bottom Row:** Standard 5-fold cross-validation; near-saturation performance ($> 99\%$) is observed with more training data. Mean and standard deviation are reported over 10 independent runs (50 total cycles).

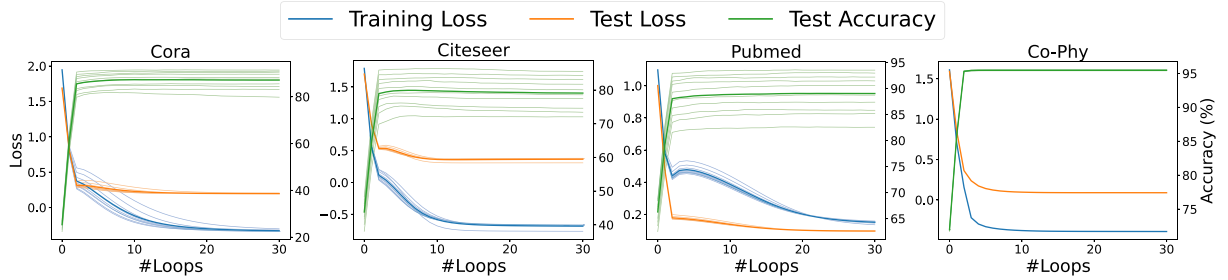


Fig. 12. The dynamics of training loss, test loss, and test accuracy of GSPNet across the co-training loops. Results from 10 experiments (refer to the caption of Fig. 11) with distinct random seeds are depicted in translucent thin curves, while their average is captured in thicker, solid curves. It is noteworthy that GSPNet's performance on Co-Phy was so consistent that the curves from different experiments almost overlap.

GSPNet and other models, as shown in the top-center plot of Fig. 10. This can be attributed to the fact that for node classification tasks, the output layer H was set to the number of categories, 8. This setting results in the dominant computational complexity of the 1-layer GSPNet ($\mathcal{O}(NDH)$) being lower than that of multi-layer GSPNet ($\mathcal{O}(ND\bar{H})$).

- **Training Time versus Number of Spectral Bases (C):** As depicted in the top-right plot of Fig. 10, when C is below 2^4 , the training time of GSPNet tends to increase with the number of layers. However, as C increases further, the training time curves of different layers overlap. This phenomenon is primarily due to the computational demands of the Riemannian gradient projection during training, which involves singular value decomposition of a matrix of size $\mathcal{O}(NC^2)$. Therefore, for larger values of C , the training time of GSPNet is predominantly determined by $\mathcal{O}(NC^2 + NCH)$, a factor independent of the number of network layers.

Concerning the memory overhead, as depicted in the bottom row of Fig. 10, it is noteworthy that the GSPNet models, regardless of their layer count, demonstrate remarkably consistent storage requirements across various configurations. This uniformity is due to the dominant storage cost factor $\mathcal{O}(ND + NC + M)$,

which remains unaffected by the number of layers. Moreover, the memory usage curves exhibit a linear increment in correlation with the increases in N , D , and C .

H. Study on Resilience to Overfitting

This section empirically evaluates the robustness of GSPNet against overfitting. Recognizing that training sets in semi-supervised graph node classification are typically much smaller than testing sets, we employed a “5-fold label-scarce cross-validation” strategy. In this setup, one fold is used for training while the remaining four serve as the testing set, thereby rigorously evaluating generalization under conditions of label scarcity. Experiments were repeated 10 times per dataset using different random seeds, yielding a total of 50 training-testing cycles.

The top row of Fig. 11 demonstrates GSPNet's consistent performance. On Cora, the average accuracy consistently exceeded 80%, with only a minor dip observed in the eighth iteration. Accuracy on Citeseer ranged between 75% and 80%, while Pubmed performance primarily fell between 80% and 90%, often nearing the upper threshold. Notably, GSPNet displayed exceptional stability on Co-Phy, where the average accuracy across all ten repetitions approximated 95% with negligible

standard deviation. These results underscore the model's adaptability to varying data splits. Additionally, we evaluated the method using standard 5-fold cross-validation (Fig. 11, bottom row); in this scenario, the increased volume of training data resulted in near-perfect performance (accuracy > 99%) across all runs.

Moreover, to delve deeper into GSPNet's resistance to overfitting, Fig. 12 visualizes the evolution of training loss and test set performance throughout the training process. The results from the ten distinct random seed experiments are shown as translucent thin curves, with their average represented by thicker solid lines. A consistent trend across various experiments is the continual decline of both training and test loss curves, converging over time. Notably, the test loss curve tends to stabilize shortly after the training loss's first inflection point, while the training loss converges later. Concurrently, the test accuracy typically shows an inverse relationship to the test loss. Particularly, GSPNet's performance on Co-Phy is remarkably stable, with curves from different experiments almost overlapping. These observations confirm GSPNet's resilience to overfitting.

VI. DISCUSSION

Discussions about the scalability of GSPNet: In contrast to classic SGCNs, GSPNet requires learning a spectral transformation instead of the adjacency-matrix-polynomial coefficients, which seems to incur much higher computation costs. However, we will show that GSPNet actually maintains a level of computational efficiency nearly equivalent to that of classic SGCNs. First, let us consider a single-layer case. Classic SGCNs have a forward computation complexity of

$$\mathcal{O}(NDH) + \mathcal{O}(SMH), \quad (50)$$

where N, D, H, S and M denote the number of hidden dimensions, samples, feature dimensions, adjacency-matrix-polynomial coefficients and non-zero elements in the adjacency matrix (usually a sparse matrix), respectively. The first and second terms of (50) are due to the feature transformation and the graph propagation steps, respectively. For GSPNet, as shown in (22), the major computational consumption stems from matrix multiplication. By leveraging the associativity of matrix multiplication, the computations can be optimally ordered as follows:

$$(\mathbf{U}(\mathbf{U}^\top(\mathbf{X}\mathbf{W}))). \quad (51)$$

leading to a forward computation complexity of

$$\mathcal{O}(NDH) + \mathcal{O}(NCH) + \mathcal{O}(MC), \quad (52)$$

where the third term is due to the graph regularizer (Cf. (22)). Noting that C is usually a small number, thus we have $SM \approx NC$ and $\mathcal{O}(SMH) \approx \mathcal{O}(NCH) > \text{rapprox} \mathcal{O}(MC)$. Comparing (52) and (50) suggests that GSPNet and classic SGCNs indeed share almost similar computation efficiency. Regarding storage overhead, from (22) it can be observed that it is necessary to store $\mathbf{U}$ and $\mathbf{W}$, leading to a complexity of $\mathcal{O}(NC + DH)$. In a multi-layer scenario, direct nesting of (22) results in significant overhead and an increased risk of overfitting due to an excess

of parameters. To address this, inspired by related work [30], [55], [66], we adopt a simplified version of multi-layer GSPNet. This version allows all network layers to share the same graph spectrum representation $\mathbf{U}$ and consolidates all graph spectral projections into a unified operation. The formalized structure is as follows:

$$\mathbf{H}^{(l)} = \begin{cases} \mathbf{X} & l = 0 \\ \sigma(\mathbf{H}^{(l-1)}\mathbf{W}^{(l)}) & 1 \leq l \leq L-1. \\ \sigma(\mathbf{U}\mathbf{U}^\top\mathbf{H}^{(L-1)}\mathbf{W}^{(L)}) & l = L. \end{cases} \quad (53)$$

Here, L denotes the number of network layers. To ease our analysis, we assume all hidden layers (including the last layer) share the same dimension $\bar{H}$. Consequently, following the analysis for the single-layer case, the computational and storage complexities are respectively $\mathcal{O}(ND\bar{H} + (L-1)N\bar{H}^2 + N\bar{C}\bar{H} + MC)$ and $\mathcal{O}(NC + D\bar{H} + (L-1)\bar{H}^2)$. Experimental results in Section V-G show that GSPNet can scale to datasets of considerable size. Nevertheless, we agree that applying GSPNet to extremely large datasets (e.g. a graph with 10^8 nodes with 10^4 classes) is still a challenging task. As a promising solution, some techniques such as graph sampling and graph coarsening are with expectation to promote the scalability of GSPNet, which will be left for future work.

Discussions about GSPNet for graph classification: The current form of GSPNet is not well-tailored for graph classification. The primary reason stems from a key design of GSPNet: the learning of graph spectrum, which entails a one-to-one correspondence with the graph itself. This aspect poses no issue for node-level tasks, where each node is considered an individual sample and different samples share the same graph structure (and thus, the same graph spectrum). By exploiting this trait, GSPNet optimizes the shared graph spectrum to transfer knowledge from training samples to test samples, achieving exceptional performance in node-level tasks. The scenario becomes markedly different for graph-level tasks. In these tasks, each sample constitutes an independent graph with varying numbers of nodes and diverse structures. As a result, GSPNet encounters difficulties in learning a uniform spectrum to adapt to different graphs with distinct node counts and structures. One potential solution to this limitation involves preserving a distinct spectrum for each graph, while concurrently learning a shared coefficient matrix that can be applied across different spectral profiles. Thus the knowledge can be transferred via the shared coefficient matrix. Nevertheless, considering that all analyses and designs in our paper are centered around node-level tasks, we leave the extension of GSPNet for graph-level tasks as an area for future research.

Discussions about the co-training of GSPNet: Classic SGCNs aim to learn a good graph spectral filter while keeping the graph spectral transformation fixed. In contrast, GSPNet allows the spectral transformation to be learned from data, which endows GSPNet with a strong potential to achieve any ground-truth partitions. However, the strong potential also exacerbates the risk of over-fitting for GSPNet, meaning that extra training strategies may be needed to alleviate this issue. To this end, we propose a modified co-training strategy to alternatively optimize the objective function of GSPNet from the graph view and

the feature view, which is empirically verified to be capable of unleashing the potential theoretically promised by GSPNet. Furthermore, the efficacy of the co-training strategy is evaluated by the experiment presented in Section V-D, showing that it can generally enhance the performance of GNNs. In particular, GSPNet obtains the most significant performance gain owing to its learnable spectral transformation design. In the future, we will make our efforts to the theoretical explanation for the working mechanism of the co-training strategy applied to GSPNet, and investigate more intrinsic model design to control the risk of GSPNet falling into over-fitting.

VII. CONCLUSION

In this work, we theoretically investigate the propagation drawback of Spectral Graph Convolutional Networks (SGCNs). We formally characterize its propagation error and show that SGCNs' propagation matrices almost always (under a mild condition) incur a non-trivial propagation error. To go beyond this drawback, we propose a Graph Spectral Projection Network (GSPNet), which proved capable of approximating an ideal propagation matrix. Empirical studies demonstrate that GSPNet can better approach the ideal propagation matrices than existing SGCNs, and thus also outperform these baselines. To conclude, this paper may provide a new perspective on the propagation design of GNNs. Some further studies are left in the future, including the promotion of the scalability of GSPNet, more intrinsic model design to control the risk of GSPNet falling into over-fitting and the extension of GSPNet for graph-level tasks.

REFERENCES

- [1] P.-A. Absil, A. Edelman, and P. Koev, "On the largest principal angle between random subspaces," *Linear Algebra Appl.*, vol. 414, no. 1, pp. 288–294, 2006.
- [2] P.-A. Absil, R. Mahony, and R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*, Princeton, NJ, USA: Princeton Univ. Press, 2009.
- [3] M. Balcilar, G. Renton, P. Héroux, B. Gaüzère, S. Adam, and P. Honeine, "Analyzing the expressive power of graph neural networks in a spectral perspective," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=qh0M9XWxnv>
- [4] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi, "Graph neural networks with convolutional ARMA filters," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3496–3507, Jul. 2022.
- [5] A. Blum and T. Mitchell, "Combining labeled and unlabeled data with co-training," in *Proc. Conf. Comput. Learn. Theory*, 1998, pp. 92–100.
- [6] D. Bo, X. Wang, C. Shi, and H. Shen, "Beyond low-frequency information in graph convolutional networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 3950–3957.
- [7] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*, Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [8] J. Bruna, W. Zaremba, A. Szlam, and Y. Lecun, "Spectral networks and locally connected networks on graphs," in *Proc. Int. Conf. Learn. Representations*, 2014. [Online]. Available: <https://openreview.net/forum?id=DQNsQf-Us0DBa>
- [9] T. T. Cai, J. Fan, and T. Jiang, "Distributions of angles in random packing on spheres," *J. Mach. Learn. Res.*, vol. 14, no. 21, pp. 1837–1864, 2013.
- [10] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1725–1735.
- [11] X. Chen, W. Hong, F. Nie, D. He, M. Yang, and J. Z. Huang, "Spectral clustering of large-scale data by directly solving normalized cut," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2018, pp. 1206–1215.
- [12] Y. Chen, I. Segovia, and Y. R. Gel, "Z-GCNETs: Time zigzags at graph convolutional networks for time series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1684–1694.
- [13] F. R. Chung and C. G. Fan, *Spectral Graph Theory (Revised and Improved)*. Providence, RI, USA: Amer. Math. Soc., 1997.
- [14] M. Coskun, A. Baggag, and M. Koyutürk, "ArnoldiGCL: Graph contrastive learning via learnable Arnoldi-based guided spectral Chebyshev polynomial filters," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2025, pp. 380–391.
- [15] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3844–3852.
- [16] R. Duan et al., "Unifying homophily and heterophily for spectral graph neural networks via triple filter ensembles," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 93540–93567.
- [17] M. Gori, G. Monfardini, and F. Scarselli, "A new model for learning in graph domains," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, 2005, pp. 729–734.
- [18] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
- [19] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [20] J. Hamm and D. D. Lee, "Grassmann discriminant analysis: A unifying view on subspace-based learning," in *Proc. Int. Conf. Mach. Learn.*, 2008, pp. 376–383.
- [21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [22] M. He, Z. Wei, Z. Huang, and H. Xu, "BernNet: Learning arbitrary graph spectral filters via Bernstein approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 14239–14251.
- [23] R. A. Horn and C. R. Johnson, *Matrix Analysis*, Cambridge, U.K.: Cambridge Univ. Press, 2012.
- [24] J. Huang, F. Nie, and H. Huang, "Spectral rotation versus k-means in spectral clustering," in *Proc. AAAI Conf. Artif. Intell.*, 2013, pp. 431–437.
- [25] K. Huang, Y. G. Wang, M. Li, and P. Liò, "How universal polynomial bases enhance spectral graph neural networks: Heterophily, over-smoothing, and over-squashing," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 20310–20330.
- [26] M. Jin et al., "Towards expressive spectral-temporal graph neural networks for time series forecasting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 6, pp. 4926–4939, Jun. 2025.
- [27] J. Jumper et al., "Highly accurate protein structure prediction with AlphaFold," *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.
- [28] K. Kim and H.-S. Oh, "Cross-spectral analysis of bivariate graph signals," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 12, pp. 11141–11151, Dec. 2025.
- [29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [30] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=SJU4ayYgl>
- [31] J. Klicpera, A. Bojchevski, and S. Günnemann, "Combining neural networks with personalized PageRank for classification on graphs," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/references/pdf?id=BJ3KbIdaX>
- [32] M. Kopp et al., "Traffic4cast at NeurIPS 2020? Yet more on the unreasonable effectiveness of gridded geo-spatial processes," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 325–343.
- [33] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3538–3545.
- [34] N. Liao, H. Liu, Z. Zhu, S. Luo, and L. V. Lakshmanan, "A comprehensive benchmark on spectral GNNs: The impact on efficiency, memory, and effectiveness," *Proc. ACM Manage. Data*, vol. 3, no. 4, pp. 1–29, 2025.
- [35] Y.-W. E. Lin, R. Talmon, and R. Levie, "Equivariant machine learning on graphs with nonlinear spectral filters," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 128182–128226.
- [36] F. Liu and Q. Wang, "Asymmetric learning for spectral graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 18798–18806.
- [37] N. Liu, X. He, T. Laurent, F. Di Giovanni, M. M. Bronstein, and X. Bresson, "A general graph spectral wavelet convolution via chebyshev order decomposition," in *Proc. Int. Conf. Mach. Learn.*, 2025, pp. 38598–38622.
- [38] K. Lu et al., "Improving expressive power of spectral graph neural networks with eigenvalue correction," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 14158–14166.
- [39] C. Morris et al., "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4602–4609.

- [40] A. Ng, M. I. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Proc. Adv. Neural Inf. Process. Syst.*, 2002, pp. 849–856.
- [41] A. Ortega, P. Frossard, J. Kovačević, J. M. Moura, and P. Vandergheynst, "Graph signal processing: Overview, challenges, and applications," *Proc. IEEE*, vol. 106, no. 5, pp. 808–828, May 2018.
- [42] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.
- [43] T. Qin, K.-J. Chen, and Z. Liu, "Enhancing spectral GNNs: From topology and perturbation perspectives," in *Proc. Int. Conf. Mach. Learn.*, 2025, pp. 50214–50234.
- [44] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [45] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 8, pp. 888–905, Aug. 2000.
- [46] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, 2008.
- [47] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=rJXMpikCZ>
- [48] U. Von Luxburg, "A tutorial on spectral clustering," *Statist. Comput.*, vol. 17, no. 4, pp. 395–416, 2007.
- [49] H. Wang and Z. Wang, "Optimal exact recovery in semi-supervised learning: A study of spectral methods and graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 51614–51649.
- [50] Q. Wang, J. Gao, and H. Li, "Grassmannian manifold optimization assisted sparse spectral clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 3145–3153.
- [51] W. Wang and Z.-H. Zhou, "A new analysis of co-training," in *Proc. Int. Conf. Mach. Learn.*, 2010, pp. 1135–1142.
- [52] X. Wang, H. Liu, C. Shi, and C. Yang, "Be confident! Towards trustworthy graph neural networks via confidence calibration," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 23768–23779.
- [53] X. Wang and M. Zhang, "How powerful are spectral graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 23341–23362.
- [54] Y. Wang, Y. Wang, J. Yang, and Z. Lin, "Dissecting the diffusion process in linear graph convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5758–5769.
- [55] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.
- [56] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu., "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [57] B. Xu, H. Shen, Q. Cao, K. Cen, and X. Cheng, "Graph convolutional networks using heat kernel for semi-supervised learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 1928–1934.
- [58] H. Xu et al., "SLOG: An inductive spectral graph neural network beyond polynomial filter," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 55348–55370.
- [59] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=ryGs6iA5Km>
- [60] Z. Yang, W. Cohen, and R. Salakhudinov, "Revisiting semi-supervised learning with graph embeddings," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 40–48.
- [61] Z. Yang et al., "WaveNet: Tackling non-stationary graph signals via graph spectral wavelets," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 9287–9295.
- [62] S. Yun, S. Kim, J. Lee, J. Kang, and H. J. Kim, "Neo-GNNs: Neighborhood overlap-aware graph neural networks for link prediction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 13683–13694.
- [63] B. Zhang, L. Zhao, and H. Maron, "On the expressive power of spectral invariant graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 60496–60526.
- [64] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 249–270, Jan. 2022.
- [65] L. Zhao and L. Akoglu, "PairNorm: Tackling oversmoothing in GNNs," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=rkecl1rtwB>
- [66] H. Zhu and P. Koniusz, "Simple spectral graph convolution," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=CYO5T-YjWZV>



Yangli-ao Geng (Member, IEEE) received the PhD degree in computer science and technology from Beijing Jiaotong University, Beijing, China, in 2021. From 2021 to 2023, he was a postdoctoral fellow with the Department of Computer Science and Technology, Tsinghua University. He is currently an assistant professor with the School of Computer Science and Technology, Beijing Jiaotong University. His research interests include graph neural networks, spectral graph methods, and graph learning.



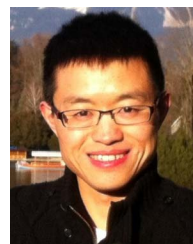
Yuxiao Dong (Senior Member, IEEE) is currently an associate professor with the Department of Computer Science and Technology, Tsinghua University. His research interests include data mining, graph representation learning, pre-training models, and social networks, with an emphasis on developing machine learning models to address problems in web-scale systems. He was the recipient of the 2017 SIGKDD Dissertation Award Honorable Mention and 2022 SIGKDD Rising Star Award.



Wenzheng Feng received the BSc degree from the Department of Computer Science and Technology, Beihang University, Beijing, China, and the PhD degree from the Department of Computer Science and Technology, Tsinghua University, Beijing, in 2023. He is currently a researcher with Huawei Beijing Research Institute. His research interests include data mining and graph neural networks.



Qingyong Li (Member, IEEE) received the BSc degree in computer science and technology from Wuhan University, Wuhan, China, in 2001, and the PhD degree in computer science and technology from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2006. He is currently a professor with Beijing Jiaotong University, Beijing. His research interests include computer vision and artificial intelligence.



Jie Tang (Fellow, IEEE) is currently a professor with the Department of Computer Science and Technology, Tsinghua University. He has authored or coauthored more than 200 research papers in major international journals and conferences. His research interests include artificial intelligence, data mining, social networks, and machine learning. He was honored with the SIGKDD Test-of-Time Award. He is a fellow of ACM.